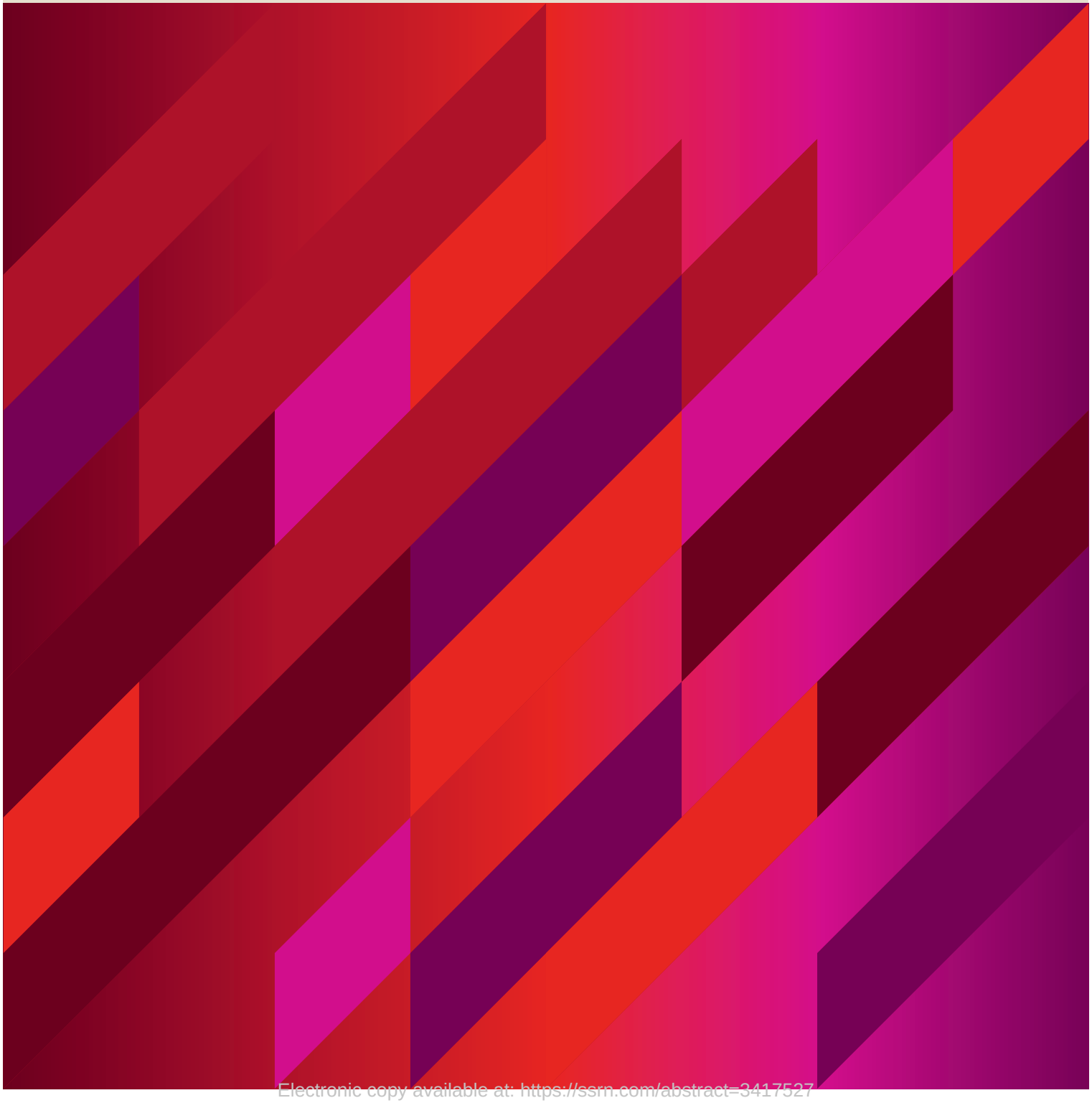
MACQUARIE
BUSINESS SCHOOL

MACQUARIE
University

Macquarie Business School

SSRN PAPER SERIES



Electronic copy available at: https://ssrn.com/abstract=3417527

Forecasting Australian Real House Price Index: A Comparison of Time Series and Machine Learning Methods[1]

George Milunovich
Macquarie University, Sydney, NSW, 2109, Australia
george.milunovich@mq.edu.au

July 4, 2019

Abstract

We employ forty-seven different algorithms to forecast Australian log real house prices and growth rates, and compare their ability to produce accurate out-of-sample predictions. The algorithms, which are specified in both single- and multi-equation frameworks, consist of traditional time-series models, machine learning (ML) procedures and deep learning neural networks. A method is adopted to compute iterated multi-step forecasts from nonlinear ML specifications. While the rankings of forecast accuracy depend on the length of the forecast horizon, as well as on the choice of the dependent variable (log price or growth rate), a few generalisations can be made. For 1- and 2-quarter ahead forecasts we find a large number of algorithms which outperform the random walk with drift benchmark. We also report several such outperformances at longer horizons of 4 and 8 quarters, although these are not statistically significant at any conventional level. Six of the eight top forecasts (4 horizons $\times$ 2 dependent variables) are generated by the same algorithm, namely a linear support vector regressor (SVR). The other two highest-ranked forecasts are produced as simple mean forecast combinations. Linear ARMA and VAR models produce accurate 1-quarter ahead predictions, while forecasts generated by deep learning nets rank well across medium and long forecast horizons.

Key Words: Forecasting, Australian Real House Price Index, Time Series, Machine Learning, Neural Networks, Autoregression
JEL classification: C53, C32, R3

[1]The author acknowledges support from the Australian Research Council Discovery Grant DP190102049.

Electronic copy available at: https://ssrn.com/abstract=3417527

1 Introduction

Residential real estate provides shelter, preserves wealth and is one of the main drivers of the Australian economy via construction and finance. Having access to accurate house price predictions is therefore equally important to central banks, financial supervision authorities, investors and home owners. In this article we explore the ability of increasingly popular machine learning (ML) and deep learning (DL) algorithms to forecast quarterly real house prices in Australia, and compare their performance against a number of traditional time series models. More specifically, we employ 47 algorithms to generate out-of-sample predictions of log house prices and growth rates across the forecast horizons of 1, 2, 4 and 8 quarters. We then measure forecast accuracy according to the mean squared error (MSE), and evaluate each forecast relative to a benchmark. Statistical tests provide further evidence regarding any such outperformance. We conclude by producing a ranking of the forecasting algorithms which is informative about their ability to produce accurate house price predictions.

Forecasting national house price indices is challenging for a number of reasons. The main issue is a relatively short length of available time series data. House price indices are typically constructed at monthly and quarterly frequencies which limits the length of computed indices, and makes model building and testing difficult. The analysis is further complicated by asymmetric boom and bust cycles that introduce nonlinear effects (Miles, 2008). As discussed in Genesove and Mayer (2001) and Engelhardt (2003) high levels of loss aversion associated with home ownership cause house prices to adjust more quickly when they rise towards equilibrium value, than when they fall. Lastly, large transaction costs prevelant in the housing markets tend to amplify any nonlinear dynamics that are present in the data (Muellbauer and Murphy, 1997).

Despite the modelling difficulties discussed above there is substantial literature on forecasting house prices. A key finding is that house price growth rates exhibit positive serial dependence and predictability over short horizons, see e.g. Case and Shiller (1989, 1990), Gau (1984, 1985), McIntosh and Henderson (1989), Schindler (2013). The evidence, however, is somewhat mixed regarding the ability of nonlinear models to outperform linear autoregressive-moving average (ARMA) specifications. For instance, Crawford and Fratantoni (2003) report that while regime-switching models outperform ARMA in-sample, they fall behind when forecasting out-of-sample. Similarly, Balcilar et al. (2015) find that smooth-transition autoregressive models fail to outperform linear AR specifications over short horizons. These findings are puzzling given that the studies mentioned above find substantial evidence of nonlinear behaviour in house prices. The literature also investigates the ability of economic factors to improve the accuracy of house price forecasts. Rapach et al. (2007) find that autoregressive distributed lag models outperform univariate autoregressions in forecasting state house price indices in the US. In a similar setting Bork and Møller (2015) report that the best forecasting model varies over time as well as across the US states. Gupta, Kabundi and Miller (2011) forecast the aggregate US real house price growth using two sets of explanatory variables – a small set containing 10 economic factors and a large set of 120 variables. They report that the small-scale model outperforms all other specifications, including the random walk. Bork and Møller (2018) employ factor analysis with

Electronic copy available at: https://ssrn.com/abstract=3417527

128 economic time series and show that macroeconomic fundamentals have strong predictive power for the US house price returns. Their model outperforms predictions generated from the price-rent ratio, autoregressive benchmarks and regression models based on smaller datasets. Further studies are surveyed in Ghysels, Plazzi, Valkanov and Torous (2013).

This paper contributes to the existing literature in several ways. First, we compare a large number of forecasting algorithms according to their accuracy in predicting Australian real house prices and growth rates. In contrast, the existing studies typically consider one model at a time and do not conduct forecast comparisons, see e.g. Bourassa and Hendershott (1995) and Abelson et al. (2005). Our predictions are generated from a set of algorithms that includes traditional time series models as well as alternative machine and deep learning specifications. This is of interest for at least two reasons. Many of the ML and DL algorithms i) have the ability to capture nonlinear relationships, and ii) employ cross-validation rather than information criteria for model selection. Whether such departures from traditional methods make ML more suitable for forecasting house prices is an open empirical question. Second, we extend the idea of multi-equation models from linear vector-autoregressions (VARs) to all of our machine learning specifications. For instance, we construct multi-equation support vector regressors and multi-equation random forests. This is needed in order to compute iterated multi-step ahead forecasts in a multivariate context. An advantage of this approach is that only one model needs to be specified in order to make predictions for all forecast horizons. Lastly, as is well known, there are complications in constructing iterated multi-step ahead predictions from nonlinear models. We address this issue by adopting a residual-based procedure of Brown and Mariano (1989) to generate unbiased multi-step forecasts from nonlinear ML and DL algorithms.

Our dataset contains an Australian real house price index collected at the quarterly frequency over the 1972:Q3 - 2017:Q3 period. In addition, we obtain data on seven economic factors that may help improve house price forecasts. These include inflation, exchange and unemployment rates, house rents, a stock price index, gross disposable income per capita, and mortgage rates. We choose the seven predictors largely in consideration of the existing literature. For instance, a large number of studies suggest that income often plays a key role in forecasting house price growth rates, see e.g. Case and Shiller (1990), Malpezzi (1999) and Favilukis et al. (2017). Similarly, rent is employed as a relevant predictor in Plazzi et al. (2010) and DiPasquale and Wheaton (1994) and interest rates in Muellbauer and Murphy (1997). In further literature inflation and equities help explain Australian house prices in Abelson et al. (2005), while employment features in Abraham and Hendershott (1996).

We setup our investigation in a fixed estimation window scheme, which facilitates comparing forecasts from nested models via the Diebold and Mariano (1995, DM) test. Indeed, in this setting the DM test is equivalent to the Giacomini and White (2006, GW) unconditional test of equal predictive accuracy. The GW conditions permit comparisons across nested and misspecified models, as well as between model estimated using different estimation procedures. This is relevant to our mix of time series, ML and DL algorithms. We train each algorithm once on the 1972:Q3 - 2004:Q4 in-sample period, and then generate forecasts over the 2005:Q1 - 2017:Q3

Electronic copy available at: https://ssrn.com/abstract=3417527

out-of-sample interval. Given the sample size this creates a rough 70:30 percent split between train and test samples. Since initial unit-root tests find evidence of nonstationarity in the log level data, we first specify and estimate forecasting algorithms on the house price growth rate which is stationary. The growth rate forecasts are then integrated into the predictions of log prices.

We attempt to summarize empirical results across 4 forecast horizons (1, 2, 4 and 8 quarters) and 2 dependent variables (growth rate and log price), thus a total of 8 forecast horizon/dependent variable configurations. The first point to note is that the forecast accuracy, relative to the random walk with drift benchmark[2], declines as the forecast horizon increases. This is in line with the early results of Case and Shiller (1989, 1990) who find evidence of short-term predictability in the US house price growth rates. The best forecast is generated by a univariate (single-equation) linear support vector regressor, denoted SVR (Lin.)^{se}. This algorithm ranks first in 6 out of the 8 forecast-horizon/dependent-variable combinations. In the case of 1- and 2-quarter ahead forecasts, SVR (Lin.)^{se} reduces the MSE by up to 45 percent. Empirical support for a similar algorithm is also found in the US context by Plakandaras et al. (2015).

Considering the results in more detail we observe that each of the top twenty 1-quarter ahead forecasts outperforms the benchmark. These outperformances are statistically significant at the 5 percent level, both individually according to the DM/GW tests and jointly as judged by the Hochberg-Bonferroni (HB) multiple hypothesis p-value bounds (Hochberg, 1988). Forecasting 2 quarters ahead yields similar results. The top 5 models across the forecast horizons of 1 and 2 quarters include simple ARMA and VAR models, as well as a more complicated multi-layer perceptrons. Forecasting 4 quarters ahead is more difficult although there are a number of models that outperform the benchmark, especially when forecasting log prices. For instance SVR (Lin.)^{se} still provides about 20 percent reduction in MSE relative to the benchmark with a p-value of 0.063, while a forecast combination outperforms with a p-value of 0.039. In contrast, there are no statistically significant reductions of the MSE when making predictions 8 quarters ahead. Thus, while there are substantial gains to be made from constructing predictive models for the forecast horizons of 1, 2, and 4 quarters, longer-term forecasts may simply rely on the random walk with drift benchmark.

Of further interest is that the top 10 forecasts are dominated by univariate (single-equation) models, although there are exceptions for longer forecast horizons. Regarding the linear vs. nonlinear specifications we obtain mixed findings. Linear models appear more suited for making 1-quarter ahead predictions, while the results are not clear cut in relation to the 2- and 4-quarter horizons. In contrast, nonlinear specifications prevail in the top 10 predictions when forecasting 8 quarters ahead. This confirms the results of Balcilar et al. (2015) for the US market.

The rest of the paper is organised as follows. Section 2 outlines our forecasting methodology and algorithms, while Section 3 presents the dataset and descriptive statistics. Empirical findings

[2]Although we refer to the random walk with drift as the benchmark model, technically this is correct only for log prices. Upon differencing the random walk we obtain the unconditional mean prediction which is used as the benchmark forecast for the growth rates.

Electronic copy available at: https://ssrn.com/abstract=3417527

are provided in Section 4. Finally, section 5 contains a number of conclusions.

2 Forecasting Methodology

We consider algorithms which form predictions about future log house prices and growth rates on the basis of available historical information. All models are specified for the growth rates r_t which are forecast first. These predictions are then integrated into the forecasts of log house prices y_t. In total there are forty-seven forecasts computed from a range of different models, ensembles and simple forecast combinations. Most of the algorithms listed below in Table 1 are specified in two forms: i) as univariate single-equation models where the dependent variable is a scalar, i.e. the house price growth rate, and ii) as multi-equation models when the dependent variable is an 8×1 vector containing the house price growth rate, as well as, seven potential explanatory variables discussed in Section 3. In this case the dependent variable is given by the vector $r_t = [r_{1t} \quad r_{2t} \quad \dots \quad r_{8t}]'$ and we have a system of eight forecasting equations.

In what follows $f_{t+h|t}$ denotes a forecast of the future scalar (or vector) variable r_{t+h} ($h \geq 1$) made at the time period t on the basis of the information set $\mathcal{F}_t = \sigma(r_t, r_{t-1}, \dots)$. This information set represents the knowledge of the history of the system, as specified by the smallest σ-field generated by the conditioning variables. In this study we employ a fixed estimation window procedure. This method implies that all models are estimated once on the in-sample (train) dataset with the index set $I_{\text{train}} = \{t|t = 1, 2, \dots, T^*\}$. Trained algorithms are then employed to construct out-of-sample forecasts over the time periods t belonging to the test dataset indexed by $I_{\text{test}} = \{t|t = T^* + 1, T^* + 2, \dots, T\}$.

Model selection is performed via information criteria for traditional time series specifications, and according to cross-validation in the case of ML and DL algorithms. In particular, we implement K-fold cross-validation by splitting the in-sample (train) dataset into $K = 10$ consecutive parts (folds). Each kth ($k = 1, 2, \dots, 10$) fold is used to evaluate the performance of models fitted on the remaining $K - 1$ folds, thus producing K estimates of the performance measure (MSE in our case), which are then averaged. Hyperparameters are then set to the values which generate the smallest average MSE and the final model is re-estimated on the entire in-sample period. Bergmeir et al. (2018) show that K-fold cross-validation works well in time series model that have uncorrelated errors. Since we set the maximum lag length to 12 quarter this is likely to be the case in our study.

2.1 Forecasting Algorithms

Table 1 lists all forecasting algorithms employed in our study. The algorithms combine an estimation method, e.g. Lasso, with a forecasting model given by a conditional mean equation, e.g. VAR(p). In the next two subsections we provide a brief description of each algorithm with the discussion organised according to whether the predictive model is a linear or nonlinear function of the information set.

Electronic copy available at: https://ssrn.com/abstract=3417527

2.1.1 ML Algorithms Estimating Linear Models

Most of the linear models discussed here, with the exception of ARMA models, can be written in the following form

$$r_{t+1} \quad = \quad c + A_1 r_t + A_2 r_{t-1} + \cdots + A_p r_{t-p+1} + u_{t+1}, \qquad \max(p) = 12. \tag{1}$$

As mentioned above r_{t+1} may either be i) a scalar representing the house price growth rate, or ii) an 8×1 vector containing relevant variables, first of which is the house price growth rate. In the first case we have an autoregression of order p denoted as AR(p), while in the second instance (1) represents a system of equations known as the vector autoregression VAR(p). In either case, the next period's value of the dependent variable/vector depends on its p past values. When (1) refers to a VAR system the parameter matrices A_i are fixed and are of dimension 8×8, while c represents a 8×1 intercept vector. The forecast error u_{t+1} is an 8-dimensional white noise process. In the AR case all of these quantities are 1-dimensional. The lag length p is the *hyperparameter* for this model, i.e. the model is estimated conditional on the value of p which is chosen by some external mechanism. We refer to the process of choosing hyperparameters as the *model selection* problem. We set the maximum lag length p considered to be 12 quarters.

Algorithms 1 and 2 given in Table 1 are two simple unconditional specifications. Model 1 assumes that future growth rates are on average zero, which is equivalent to the random walk model in log prices. This model is obtained by setting $c = 0$ and $A_1 = \cdots = A_p = 0$ in (1). Model 2 adds a drift to the random walk specified for log prices, thus transforming the model for the growth rate into the simple unconditional mean specification. This model is given by restricting $A_1 = \cdots = A_p = 0$, while estimating the intercept c. Model 2 forms our *benchmark* forecast for both log prices and growth rates. Although we refer to it as the random walk with drift throughout the paper, this is technically true only when forecasting log prices.

Specifications 3 and 4 are univariate autoregressive moving average ARMA(p, q) models. We specify them as $\phi(L) r_{t+1} = c + \theta(L) u_{t+1}$ where AR(p) and MA(q) lag polynomials are given by $\phi(L)$ and $\theta(L)$, respectively. Algorithm 3 sets $(p, q) = (1, 1)$ given that ARMA(1, 1) has been shown to perform well in a variety of forecasting situations. In contrast, Algorithm 4 performs model selection, i.e. choosing the values of (p, q) via the Bayesian Information Criterion (Schwarz, 1978). ARMA models are implemented with `tsa.statespace.SARIMAX` class from the `StatsModels` library in Python.

VAR(p) is the first multi-equation specification listed in Table 1. It is a vector autoregression of order p given in equation (1). Model selection, i.e. the determination of p, is performed using the Bayesian Information Criterion (BIC). The algorithms listed under item 6 are single- and multi-equation (vector) autoregressions where the lag length is set to its maximum value of 12, written respectively as AR(12) and VAR(12). VAR models are fitted and forecasted using `tsa.api.VAR` class from `StatsModels`, while the AR(12) model is implemented using the same library employed for ARMA.

Methods 7, 8, and 9 have the same basic form as in (1) but differ from standard AR(p)/VAR(p) models in the estimation method, and hence in the content of the RHS variables. Namely, the

Electronic copy available at: https://ssrn.com/abstract=3417527

Table 1: Forecasting Althorithms Specified for House Price Growth Rates (r_t)

	Forecasting Algorithm	Single-/Multi- Eq.	Label	Model Selection
1	No Change	SE	AR(0)	N/A
2	Historical Average	SE	AR(0)-C	N/A
3	ARMA$(1,1)$	SE	ARMA$(1,1)$	N/A
4	ARMA(p,q)	SE	ARMA(p,q)*	BIC
5	VAR(p)	ME	VAR(p)*	BIC
6	AR(12)/VAR(12)	SE & ME	AR(12)/VAR(12)	N/A
7	Ridge	SE & ME	Ridge$^{se/me}$	K-Fold
8	Lasso	SE & ME	Lasso$^{se/me}$	K-Fold
9	Elastic Net	SE & ME	ElasticNet$^{se/me}$	K-Fold
10	Support Vector Machine	SE & ME	SVR$^{se/me}$	K-Fold
11	Support Vector (Polynomial)	SE & ME	SVR (Poly)$^{se/me}$	K-Fold
12	Support Vector (RBF)	SE & ME	SVR (RBF)$^{se/me}$	K-Fold
13	Support Vector (Sigmoid)	SE & ME	SVR (Sig)$^{se/me}$	K-Fold
14	Kernel Ridge (Polynomial)	SE & ME	KRR (Poly)$^{se/me}$	K-Fold
15	Kernel Ridge (RBF)	SE & ME	KRR (RBF)$^{se/me}$	K-Fold
16	Kernel Ridge (Sigmoid)	SE & ME	KRR (Sig)$^{se/me}$	K-Fold
17	KNeighbors	SE & ME	KNeighbors$^{se/me}$	K-Fold
18	DecisionTree	SE & ME	DecisionTree$^{se/me}$	K-Fold
19	MElayer Perceptron	SE & ME	MLP$^{se/me}$	K-Fold
20	LSTM Recurrent NN	SE & ME	LSTM$^{se/me}$	K-Fold
21	RandomForest	SE & ME	DecisionTree$^{se/me}$	K-Fold
22	Gradient Boost	SE & ME	GradientBoost$^{se/me}$	K-Fold
23	XG Boost	SE & ME	XGB$^{se/me}$	K-Fold
24	Light GBM	SE & ME	LGBM$^{se/me}$	K-Fold
25	Forecast Combination: Mean	SE & ME	Comb. Means$^{se/me}$	N/A
26	Forecast Combination: Median	SE & ME	Comb. Median$^{se/me}$	N/A

Notes: SE=single-equation specifications; ME=multi-equation specifications. Model selection refers to the estimation of hyperparameters which is performed either via Bayesian Information Criterion (BIC) or via K-Fold Cross Validation.

Ridge (Hoerl and Kennard, 1970), Least Absolute Shrinkage and Selection Operator – Lasso (Tibshirani, 1996) and ElasticNet (Zou and Hastie, 2005) need not include all of $r_t, r_{t-1}, \ldots, r_{t-p+1}$ on the RHS of (1). Instead any subset of these variables may be selected by the K-Fold cross-validation procedure described above. The difference between algorithms 7, 8 and 9 is in the type of regularization they implement, that is the kind of penalty function that is imposed on extreme parameter values. Ridge, Lasso and ElasticNet estimate the parameters of (1) by minimizing the sum of least squares residuals subject to different penalties, which lead to the following objective functions

1. Ridge (L2 penalty): $\sum_t (y_{t+1} - f_{t+1|t})^2 + \lambda_1 \sum_{j=1}^{n_a} a_j^2$;
2. Lasso (L1 penalty): $\sum_t (y_{t+1} - f_{t+1|t})^2 + \lambda_1 \sum_{j=1}^{n_a} |a_j|$;

Electronic copy available at: https://ssrn.com/abstract=3417527

3. ElasticNet (L1 + L2 penalty): $\sum_t (y_{t+1} - f_{t+1|t})^2 + \lambda_1 \sum_{j=1}^{n_a} |a_j| + \lambda_2 \sum_{j=1}^{n_a} a_j^2$.

The penalties are implemented on the set $\{a_j, 1 \leq j \leq n_a\}$ which collects the parameters across all A_i matrices in (1). The hyperparameters $\lambda_1 \geq 0$ and $\lambda_2 \geq 0$ control the strength of penalty, and are also chosen by the K-Fold cross-validation. Larger values of these parameters lead to greater penalty and a stronger effect of parameter shrinking towards zero. Setting $\lambda_1 = 0$ and/or $\lambda_2 = 0$ leads to the standard OLS estimator. The absolute value penalty in Lasso leads to many corner solutions for which the parameter estimates $\hat{a}_j$ are equal to zero. The algorithms differ along two further dimensions. First, while the Lasso penalty is insensitive to the choice among a set of strongly correlated predictors, Ridge will shrink the coefficients of correlated regressors towards each other. ElasticNet on the other hand, encourages a grouping effect, where strongly correlated predictors tend to be in or out of the model together. Furthermore, Ridge and ElasticNet are well suited as variable selection methods when the number of predictors (n_a) is greater than the number of observations (T^*), while Lasso can select at most T^* variables. Our implementations use `linear_model.ElasticNet`, `linear_model.Lasso`, and `linear_model.Ridge` classes from the Scikit-learn library `sklearn`.

Method 10 is the Support Vector Machine for regression (SVR). It estimates the parameters of (1) by minimizing the following function

$$\sum_t V_\xi(y_{t+1} - f_{t+1|t}) + \frac{\lambda_1}{2} \sum_{j=1}^{n_a} a_j^2$$

where

$$V_\xi(u) = \begin{cases} 0 & \text{if} \quad |u| < \xi \\ |u| - \xi & \text{otherwise.} \end{cases}$$

Thus, the SVR ignores forecast errors of size smaller than ξ and in addition implements regularization via the L2 penalty (given by the second term in the objective function). The two hyperparameters ξ and λ_1 are chosen by cross-validation. We use `sklearn.svm.SVR` class to implement SVR.

2.1.2 ML Algorithms Estimating Nonlinear Models

The linear model (1) may be extended to a nonlinear function $\phi(.)$ of the data as follows

$$\begin{aligned} r_{t+1} &= \phi(r_t, r_{t-1}, \ldots, r_{t-p+1}) + u_{t+1} \\ &= \phi(R_t) + u_{t+1}, \end{aligned} \tag{2}$$

where R_t is a vector collecting conditioning variables, i.e. $R_t = [r_t' \quad r_{t-1}' \quad \ldots \quad r_{t-p+1}']'$.

While the Ridge and SVR methods discussed above estimate the linear model (1), they can be combined with the kernel method (trick) in order to estimate a version of (2). The basic idea is to augment R_t with nonlinear transformations of the elements it contains, and then to employ linear methods in this new space. Since Ridge and SVR have objective functions where R_t enters only through an inner product term, estimating (2) amounts to replacing $R_\tau' R_k$ with

Electronic copy available at: https://ssrn.com/abstract=3417527

$\phi(R_\tau)'\phi(R_k)$ in the objective functions. Kernel methods specify such inner products directly in terms of kernels $k(R_\tau, R_k)$, i.e. $k(R_\tau, R_k) = \phi(R_\tau)'\phi(R_k)$. It is possible to construct very complicated kernels that generate high-dimensional or even infinite-dimensional feature maps $\phi(R_t)$. Thus, choosing a kernel corresponds to implicitly choosing a corresponding nonlinear and high-dimensional model (2). The effect of the kernel trick is that it is possible to perform operations in the feature space in terms of the kernel values. The term 'trick' refers to the fact that if one specifies a high- or infinite-dimensional $\phi(R_t)$ directly, then the estimation problem will have correspondingly high- or infinite-dimensional solutions. However, respecifying the problem in terms of the kernel function yields finite-dimensional solutions for such high- or infinite-dimensional estimation problems. For more details see e.g. Hastie et al. (2009, pp. 167-174).

Specifications 11 – 16 implement kernel methods via SVR and Ridge, which in this setting is called Kernel Ridge Regressor (KRR). We employ three popular kernels with both of these methods, namely

1. Polynomial: $k(R_\tau, R_k) = (\gamma R_\tau' R_k + r)^d$ where d is the degree of the polynomial;
2. Radial Basis Function (RBF): $k(R_\tau, R_k) = e^{-\gamma||R_\tau - R_k||^2}$;
3. Sigmoid: $k(R_\tau, R_k) = \tanh(\gamma R_\tau' R_k + r)$.

The parameters γ and d are trained via cross-validation, while r is set to 1. The SVR class used in Python is the same as the one provided for the linear SVR (but now specified with a different kernel function), while the KRR estimator is available via the `sklearn.kernel_ridge.KernelRidge` class.

K-nearest neighbors (KNeighbors) algorithm also estimates a version of (2) and works as follows. For a number K and a distance metric, the prediction $f_{t+1|t}$ of r_{t+1} for $t \in I_{\text{test}}$ is formed by a local interpolation of the targets associated with its K nearest neighbors contained in the train set. More specifically, KNeighbors finds the set of K vectors $\mathcal{N} = \{R_i | i = 1, 2, \ldots, K; i \in I_{\text{train}}\}$ which are closest to $R_t \in I_{\text{test}}$. The forecast is then computed by averaging the corresponding targets r_{i+1} as $f_{t+1|t} = \sum_{i \in \mathcal{N}} w_i r_{i+1}$, where the weights w_i are given by the distance from R_t to each $R_i \in \mathcal{N}$, according to the metric used. The hyperparameters are K and the distance measure. Scikit-learn class for this method is `sklearn.neighbors.KNeighborsRegressor`.

A Decision Tree for regression (regression tree) is built top-down from a root node and involves partitioning the data into smaller and smaller subsets that contain similar values. The final result is a tree that splits data according to internal nodes, which have two or more branches, and leaf nodes that are terminal and provide predictions. In regression problems the algorithm uses within-node variance to split the data according to the features (the elements of R_t in (2)) that result in the highest variance reduction in r_{t+1}. The splitting of the data is performed recursively, typically using inequalities, until some termination criterion is reached. The final prediction is the within-node sample mean $\hat{\mu}$ for each leaf node. If a tree partitions the input space into K disjoint regions $G_1, G_2, \ldots, G_K$ and predicts $\hat{\mu}_k$ in region k, the final prediction is given by $f_{t+1|t} = \sum_{k=1}^{K} \hat{\mu}_k 1_{G_k}(R_t)$ with the indicator function $1_{G_k}(R_t) = 1$ if $R_t \in G_k$ and zero

Electronic copy available at: https://ssrn.com/abstract=3417527

otherwise. The hyperparameters include the maximum depth of the tree, the maximum number of features (RHS variables) to consider when looking for the best split, and the minimum number of observations required to split an internal node. These are trained via cross-validation. We used `sklearn.tree.DecisionTreeRegressor` class to deploy this algorithm.

Multilayer Perceptron (MLP) is a feed-forward neural network where the input layer, the hidden layers and the output layer are fully connected. The term feedforward implies that each layer acts as an input to the next layer without loops. The universal approximation theorem (Cybenko, 1989; Hornik et al., 1989) states that a feed-forward network with a single hidden layer containing a finite number of neurons (see q below in (3)) can approximate any Borel measurable function to any desired degree of accuracy.

In order to construct a multi-equation algorithm for MLP we build an equation for each element r_{it+1} of the vector r_{t+1} as follows

$$r_{it+1} = c_i + \sum_{j=1}^{q} \beta_{ij} \phi \left(e_i^{'} \sum_{k=0}^{p-1} A_{jk} r_{t-k} \right) + u_{it+1}, \qquad \text{for} \quad i = 1, 2 \ldots, 8. \tag{3}$$

In relation to (1) we see that the sum indexed by k represents a VAR(p) while the unit vector e_i, e.g. $e_2 = [0 \quad 1 \quad 0 \ldots 0]^{'}$, selects the row (equation) i of the VAR(p). The hidden layer is characterised by an activation function $\phi(.)$, which in our case is chosen by cross-validation from the set $\{\text{relu}, \tanh, \text{identity}\}$. Finally, q represents the size of the hidden layer which, along with the momentum for the gradient descent update, is trained via cross-validation. The equation is completed with scalar quantities β_{ij}, c_i and u_{it+1}. This algorithm is implemented via the `sklearn.neural_network.MLPRegressor` class in Python where the remaining hyperparameters left unchanged at their default scikit-learn settings.

In contrast to the MLP discussed above, a recurrent neural network (RNN) has hidden layers which get their inputs from both the input layer, i.e. historical data, as well as from the hidden layer lagged one period. If we denote the hidden layer in (3) as $\phi_{ijt} = \phi \left(e_i^{'} \sum_{k=0}^{p-1} A_{jk} r_{t-k} \right)$ then an RNN hidden layer is extended in the following way

$$\phi_{ijt} = \phi \left(e_i^{'} \sum_{k=0}^{p-1} A_{jk} r_{t-k} + \sum_{j=1}^{q} \alpha_j \phi_{ijt-1} \right), \tag{4}$$

where α_js represent scalar weights given to neurons lagged one period.

Our version of RNN is a long short-term memory (LSTM) network proposed in Hochreiter and Schmidhuber (1997) as a solution to the vanishing gradient problem. We implement the LSTM network with two hidden layers and use the same activation function as in MLP above. Lastly, we estimate LSTM with TensorFlow and Keras libraries using the `keras.models.Sequential` and `keras.layers.LSTM` classes, cross-validating the number of epochs and neurons.

Electronic copy available at: https://ssrn.com/abstract=3417527

2.1.3 Ensembles and Forecast Combinations

Random Forests (Breiman, 2001) are ensembles that fit a number of decision trees on sub-samples of the dataset and use averaging across leaf nodes to reduce model variance. Each tree is built from d features drawn randomly from n_a elements of R_t in (2) which helps control over-fitting. The default value used in scikit-learn implementation is $d = \sqrt{n_a}$. Other hyperparameters include all the hyperparameters associated with a single decision tree, as discussed above, and the number of trees to build. More trees usually lead to better forecasting performance but also to greater computational cost. Scikit-learn class `sklearn.ensemble.RandomForestRegressor` provides an implementation of this method.

Gradient Boosting (Friedman, 2001) is another type of tree ensemble where instead of generating independent trees, like in a random forest, each tree is made conditional on previous trees. It seeks to minimize some risk function $\mathbb{E}[L(r_{t+1}, f_{t+1|t})]$ through an iterative procedure. The process starts with a constant forecast $f^0_{t+1|t} = c$, e.g. a sample mean $c = \hat{\mu}$, and defines a recursion $f^k_{t+1|t} = f^{k-1}_{t+1|t} + h^k(R_t)$. This procedure is repeated for $k = 1, 2, \ldots, K$ terminating in the final forecast $f^K_{t+1|t}$. The function $h^k(R_t)$ is a regression tree fitted to the residual from the previous step, i.e. $r_{t+1} - f^{k-1}_{t+1|t} = h^k(R_t) + \varepsilon^k_{t+1}$ with some error ε^k_{t+1}. The algorithm name refers to the fact that when the loss function is given by the squared loss $L(r_{t+1}, f_{t+1|t}) = (r_{t+1} - f_{t+1|t})^2$ the residual is equal to the negative of the gradient of the loss function w.r.t. $f_{t+1|t}$. We employ a version of gradient boosting provided in the `sklearn.ensemble.GradientBoostingRegressor` class.

XGBoost is a gradient boosting algorithm which incorporates regularization of the objective function, thus providing a formal approach to the problem of over-fitting and tree pruning. It is included in our set of algorithms as it has a good track record in the data science community. For instance, as reported in Chen and Guestrin (2016) XGBoost features in 17 winning solutions out of 29 different Kaggle competitions. The `xgboost` Python library provides `XGBRegressor` class which implements this algorithm. Another free algorithm is LightGBM which is a new gradient boosting procedure provided by Microsoft. The main difference between LightGBM and XGBoost is in their approaches to computing splitting rules, which is a relatively technical issue that will not be discussed here. `lightgbm` library has the `LGBMRegressor` class used in this paper.

Lastly we compute simple forecast combinations in two ways: i) mean forecasts, and ii) median predictions. These are respectively listed as items 25 and 26 of Table 1. Mean and median combinations are calculated across all univariate models, denoted Comb. Means^{se} and Comb. Median^{se} as well as across all multivariate models which are labelled as Comb. Means^{me} and Comb. Median^{me}.

2.2 Constructing the Forecasts

As discussed at the start of this section, forecasts $f_{t+h|t}$ of the vector (scaler in the case of single-equation models) r_{t+h} are computed on the basis of the information set $\mathcal{F}_t = \sigma\{r_t, r_{t-1}, \ldots\}$. For $h = 1, 2, 4, 8$ these predictions correspond to forecasts of one, two, four and eight quarters ahead.

Electronic copy available at: https://ssrn.com/abstract=3417527

We choose to construct multi-step forecasts using an iterated rather than direct[3] method. This decision is based on several considerations. First, the iterated procedure requires only one estimated model, which can then be employed to compute forecasts for any h. In contrast, a direct procedure needs a different model for each h. This is of significance in the context of the ML algorithms employed here, some of which take substantial time to train. Second, there is the issue of effectiveness of cross-validation when applied to direct forecast methods. Namely, direct forecasts of r_{t+h} introduce an MA$(h-1)$ process into the error term, even when the true underlying errors are serially uncorrelated. This serial correlation makes cross-validation techniques problematic and may lead to a systematic underestimation of the error (Bergmeir et al., 2018). In order to avoid such issues we implement iterated multi-step forecasts. Finally, while iterated methods make efficient use of data, direct procedures are more robust to misspecification (Elliott and Timmermann, 2016, pp. 153-154). Since our dataset contains only 181 quarterly observations we prioritise on the basis of efficiency.

2.2.1 Forecasting with Linear Models

Section 2.1.1 discusses a number of ML algorithms which estimate a linear form of the conditional mean model, for instance the AR/VAR models in (1). Re-writing this equation for the time period $t+h$ we obtain

$$r_{t+h} = k + \sum_{j=0}^{p-1} \phi_j r_{t-j} + \sum_{i=1}^{h} \theta_i u_{t+i}, \tag{5}$$

where k, ϕ_js and θ_is are functions of the original parameters c and $A_1, A_2, \ldots, A_p$. Since errors aggregate linearly in (5) a multi-step prediction is given by

$$f_{t+h|t} = k + \sum_{j=1}^{p-1} \phi_j r_{t-j},$$

under the assumption that $\mathbb{E}[u_{t+i}|\mathcal{F}_t] = 0$ for $i = 1, 2, \ldots, h$.

2.2.2 Nonlinear Models

The algorithms listed in Sections 2.1.2 and 2.1.3 estimate nonlinear conditional mean equations. Iterating (2) forward produces the following set of equations

$$\begin{aligned}
r_{t+1} &= \phi(r_t, r_{t-1}, \ldots, r_{t-p+1}) + u_{t+1} \\
r_{t+2} &= \phi(\phi(r_t, r_{t-1}, \ldots, r_{t-p+1}) + u_{t+1}) + u_{t+2} \\
&\vdots \\
r_{t+h} &= \phi(\ldots\phi(\phi(\phi(r_t, r_{t-1}, \ldots, r_{t-p+1}) + u_{t+1}) + u_{t+2}) + u_{t+3})\cdots + u_{t+h-1}) + u_{t+h}.
\end{aligned} \tag{6}$$

[3] A direct forecast for any h is computed by projecting r_{t+h} directly onto the relevant conditioning variables available at time t.

Electronic copy available at: https://ssrn.com/abstract=3417527

Even when $\mathbb{E}[u_{t+i}|\mathcal{F}_t] = 0$ for $i = 1, 2, \ldots, h$, the forecasts of the above equations are complicated by the fact that the error terms u_{t+i} are inside nonlinear functions $\phi(.)$. The only exception is in the case of the 1-step ahead forecast when

$$f_{t+1|t} = \mathbb{E}[r_{t+1}|\mathcal{F}_t] \quad = \quad \phi(r_t, r_{t-1}, \ldots, r_{t-p}). \tag{7}$$

Nonlinear multi-step forecasting problems are studied by Brown and Mariano (1984) in the framework of static systems of equations. Their work is extended to dynamic nonlinear systems in Brown and Mariano (1989), where they propose a number of asymptotically unbiased predictors. In this paper we adopt their residual-based bootstrap predictor which may be described as follows

1. Randomly draw with replacement vectors $u_1^b, u_2^b, \ldots, u_{h-1}^b$ for $b = 1, 2, \ldots, B$, from the in-sample empirical distribution of u_{t+1};
2. Compute $f_{t+h|t}^b = \phi(\ldots\phi(\phi(\phi(r_t, r_{t-1}, \ldots, r_{t-p+1}) + u_1^b) + u_2^b) + u_3^b)\cdots + u_{h-1}^b)$;
3. Create forecast $f_{t+h|t} = \frac{1}{B}\sum_{b=1}^{B} f_{t+h|t}^b$ for $h \in \{2, 4, 8\}$.

We implement the residual-based predictor with $B = 100$. In addition, we also recompute the forecasts using the Monte Carlo and deterministic predictors[4] proposed in Brown and Mariano, which yield similar results. Hence the findings reported in this paper are based on the above described algorithm although further results for the other two approaches are available upon request.

2.2.3 Forecasting Log Levels

Once the forecasts of the growth rates r_{t+h} are computed they are integrated into the predictions of the I(1) log price y_{t+h}. Given the relationship $r_{t+1} = y_{t+1} - y_t$, we obtain the log price predictions $F_{t+h|t}$ from the growth rate forecasts $f_{t+h|t}$ in the following way

$$F_{t+h|t} = y_t + \sum_{i=1}^{h} f_{t+i|t}. \tag{8}$$

This provides the relationship between the forecast errors for log prices (ε_{t+h}) and the errors for growth rates (u_{t+h}) as follows $\varepsilon_{t+h} = y_{t+h} - F_{t+h|t} = [y_t + r_{t+1} + \cdots + r_{t+h}] - [y_t + f_{t+1|t} + \cdots + f_{t+h|t}] = \sum_{i=1}^{h} u_{t+i}$. As can be seen the h-step ahead forecast error in the log-level series is the sum of first h log-return forecast errors.

[4]The deterministic predictor sets all forecast errors to zero, while the Monte Carlo approach draws forecast error vectors from a simulated multivariate density, e.g. multivariate normal calibrated to in-sample error parameters. Theoretically the deterministic predictor is asymptotically biased, while the Monte Carlo and the bootstrap predictors are unbiased. The bootstrap approach is inefficient relative to the Monte Carlo method. Nevertheless, as suggested in Brown and Mariano (1989), the residual-based predictor may still be preferred to the Monte Carlo method since it is robust against mis-specifications of the error distribution.

Electronic copy available at: https://ssrn.com/abstract=3417527

2.3 Forecast Evaluations

Evaluations of predictive accuracy are conducted on the basis of computed errors for forecast horizons $h = 1, 2, 4, 8$ and forecast algorithms $\ell = 1, 2, \ldots, 47$. The errors for house price growth rates and log levels are defined as $u_{\ell,t+h} = r_{t+h} - f_{\ell,t+h|t}$ and $\varepsilon_{\ell,t+h} = y_{t+h} - F_{\ell,t+h|t}$, respectively. While most of the discussion in this subsection is framed in terms of growth rates, the same methodology is applied to log-level prices in later sections.

We perform forecast evaluations on the basis of three measures: i) relative mean squared errors (MSE), ii) Diebold and Mariano (1995, DM)/Giacomini and White (2006, GW) unconditional predictive ability test, and Hochberg-Bonferroni (HB) multiple hypothesis p-value bounds (Hochberg, 1988).

The mean squared errors are computed for each forecast algorithm ℓ and horizon h as follows

$$\text{MSE}_{\ell,h} = \frac{1}{T-h} \sum_{t=1}^{T-h} u_{\ell,t+h}^2.$$

As mentioned in Section 2.1.1 our benchmark is the random-walk with drift model for house price log levels, which amounts to unconditional historical average model for the growth rates. We denote the benchmark mean squared error as $\text{MSE}_{B,h}$. The relative MSEs are then obtained as ratios of each algorithm's MSE to the MSE of the benchmark model

$$\text{MSE}_{\ell,h}^R = \frac{\text{MSE}_{\ell,h}}{\text{MSE}_{B,h}}. \tag{9}$$

These are the magnitudes presented in Table 3 and in later tables.

In addition to computing relative MSEs we also conduct formal tests of equal predictive performance. In particular, for each forecast horizon we test the null of no performance difference against the one-sided alternative that each algorithm ℓ outperforms the benchmark algorithm B

$$H_0 : \mathbb{E}[\Delta_{\ell,h}] = 0 \quad \text{vs.} \quad H_1 : \mathbb{E}[\Delta_{\ell,h}] > 0, \tag{10}$$

where $\Delta_{\ell,h} = u_{B,t+h}^2 - u_{\ell,t+h}^2$ and $B \neq \ell$.

This type of test was first proposed in Diebold and Mariano (1995) and extended in further literature, see e.g. West (1996), Clark and McCracken (2001), Clark and West (2007). Given the fixed estimation window setup we implement here, the DM test also coincides with the unconditional predictive accuracy test of Giacomini and White (2006). One apparent drawback of the DM procedure is that the long-run variance of $\Delta_{\ell,h}$ converges to 0 when comparing forecasts generated by nested models, as we have here in some cases. A simple solution to this problem is given in Giacomini and White who propose preserving the finite-sample properties of the estimators asymptotically, rather than assuming the convergence of parameters to their limits. By retaining the effect of estimation errors, this approach effectively justifies the validity of the DM procedure even in the case of nested models with estimated parameters. Additionally, the GW method allows comparisons of forecasts generated from misspecified models and models estimated via alternative estimation procedures, which are relevant from our perspective. Thus, we refer to the tests of the hypotheses in (10) as the DM/GW procedure.

Electronic copy available at: https://ssrn.com/abstract=3417527

Lastly, since (10) is tested for each forecast algorithm we also compute a p-value bound to control for Type I error in the presence of multiple tests. In particular, we implement a method proposed in Hochberg (1988). The Hochberg-Bonferroni (HB) bound is computed as

$$\text{HB} = \min_{j=1,2,\ldots,K} (K - j + 1)p(j), \tag{11}$$

where the $p_{(1)}, p_{(2)}, \ldots, , p_{(K)}$ are the p-values associated with the DM/GW tests of (10) listed in ascending order. The HB value provides an upper p-value bound for testing that all pairwise comparisons are zero, i.e. that the null hypothesis in (10) is true for all algorithms.

3 Data

Our dataset consists of quarterly time series collected for the 1972:Q3 – 2017:Q3 time period and totalling 181 observations. The main variable of interest is the Australian Real House Price Index. This is the same Australian Department of Treasury housing index used in Abelson et al. (2005), which we have updated to present time by splicing it with a weighted average of real house prices for the six capital cities obtained from Corelogic. In addition, we obtain data on seven potential predictors including: i) consumer price index – CPI, ii) real gross disposable income per capita – Real GDI (p.c.), iii) All Ordinaries share price index deflated by the CPI – Real Equities, iv) rent on housing deflated by the CPI – Real Rent, v) unemployment rate – Unemployment Rate, vi) mortgage rate less inflation – Real Mortgage Rate, and vii) Australian dollar exchange rate as measured by the trade weighted index – Australian $ (TWI). All of these variables have been employed as predictors of house prices in the literature, as discussed in Section 1.

Figure 1 provides a first glimpse at our dataset. From the top panel, where we have five strongly trending variables, we observe that the house price index, rent, GDI, and equities move in close unison over the forty-five year period. CPI follows a similar trend, although it grows at a faster rate in the first half of the sample period and then exhibits a slower growth rate from about 1990. The other four variables exhibit relatively consistent, albeit substantially volatile, growth rates and catch up with the CPI by the end of the sample. The bottom panel of Figure 1 presents the remaining three predictors which do not display strong upward dynamics. The Unemployment Rate and the Mortgage Rate appear to move in tandem, while the TWI[5] displays an inverse relationship with the two.

Given the non-stationary behaviour of the series, which is confirmed below in Table 2, we transform the data into growth rates (log returns) as follows:

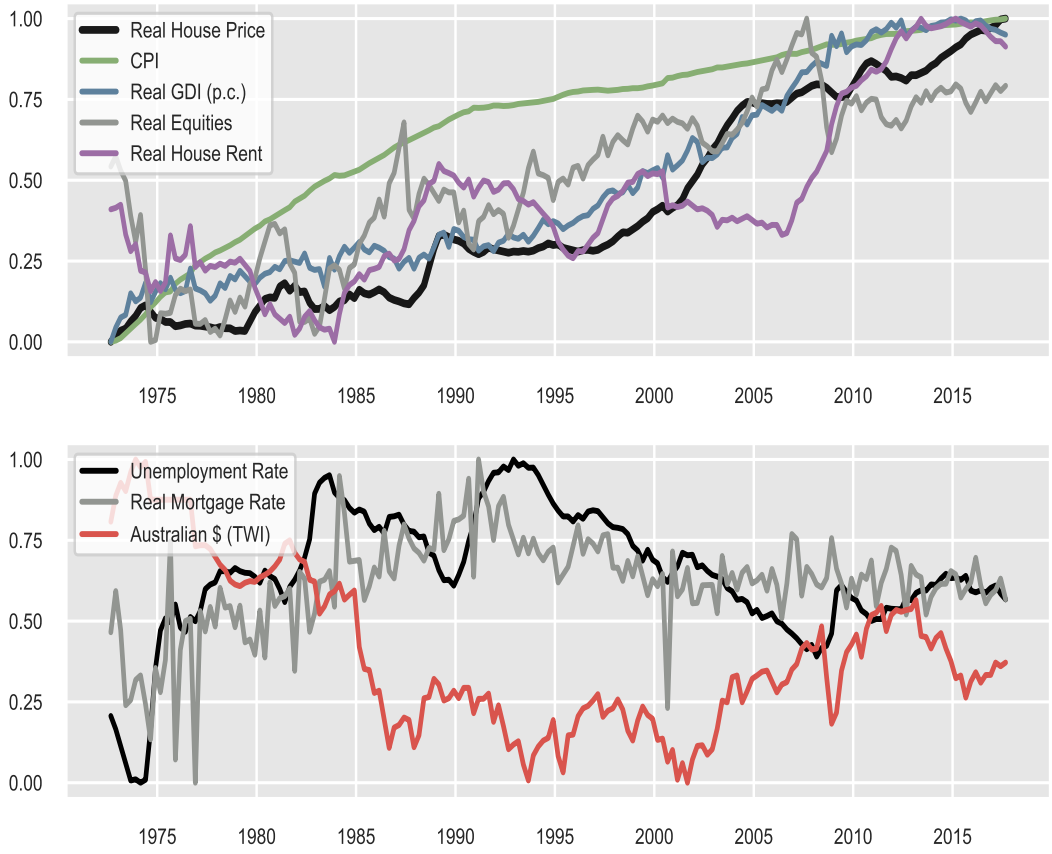
$$r_{it} = 100 \times [\ln(y_{it}) - \ln(y_{it-1})] \qquad \text{for} \quad i \in \{1, 2, \ldots, 8\}.$$

The only exception to the above rule is the Real Mortgage Rate variable which takes on several negative values. In this case we induce stationarity using the simple difference, rather than log difference, transformation, i.e. $r_{it} = y_{it} - y_{it-1}$ for the Real Mortgage Rate.

[5]The TWI expresses the value of the Australian dollar (AUD) as the amount of foreign currency per one unit of AUD. Thus, the period since the early 2000s corresponds with the strengthening of the AUD.

Electronic copy available at: https://ssrn.com/abstract=3417527

Figure 1: Historical Data Series (1972:Q3 – 2017:Q3)



Notes: The above figure depicts series scaled to the $(0, 1)$ interval using the transformation $y = \frac{x - min(x)}{max(x) - min(x)}$ This does not change the underlying trends in the data while aiding visual comparisons. All subsequent analyses are conducted on the original data.

Table 2 below provides some descriptive statistics, as well as the Dickey and Fuller (1979) unit-root test. The first four columns are computed on the growth rate series, while the ADF tests are presented for the growth rates first and then for log prices in the last column.
A number of observations are of interest. For instance, the CPI grew at an average rate of 1.281 percent per quarter over the entire sample period. This exceptionally high inflation rate is mainly due to the first part of the sample, as discussed above. Since the mid-1990s the inflation has moderated and averaged about 2.4 percent per annum. The real house price growth rate is also high, averaging about 0.812 percent per quarter. In contrast, risk associated with house price growth rates is moderate relative to that of the unemployment rate, as well as when compared to the volatilities of the real mortgage and exchange rates. Another noteworthy point is the attractiveness of Australian real estate over equities in terms of higher expected return and lower risk. Although the Australian dollar appears to have a negative mean quarterly return, this is largely due to a high starting value of the AUD in the 1970s. The last two columns of the table indicate that the data appears to be nonstationary in log levels but stationary in growth rates. at the 5 percent significance level.

Electronic copy available at: https://ssrn.com/abstract=3417527

Table 2: Summary Statistics

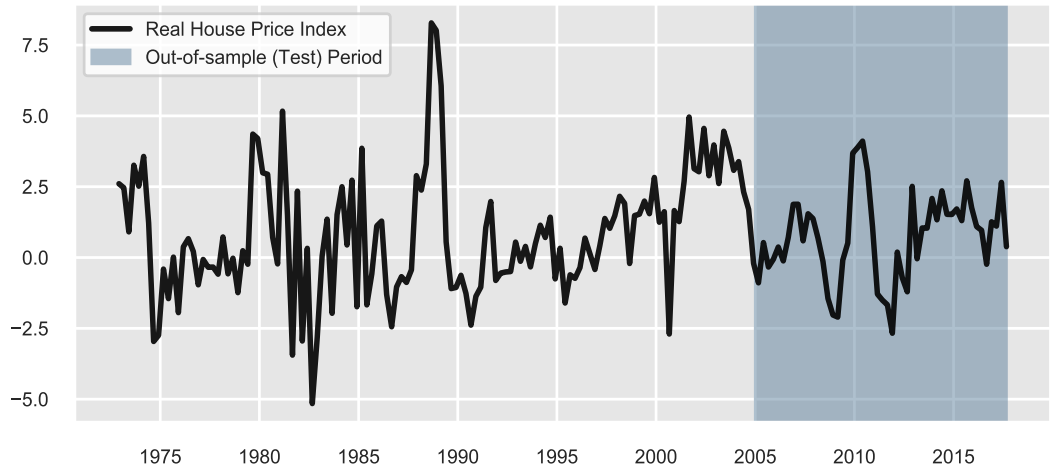
	Mean	Std.	Min. $100 \times \Delta\ln(y_t)$	Max.	ADF p-value $\Delta\ln(y_t)$	ADF p-value $\ln(y_t)$
Real House Price	0.812	2.034	-5.153	8.281	0.000	0.977
Australian $ (TWI)	-0.223	4.684	-16.115	11.972	0.000	0.340
Consumer Price Index	1.281	1.144	-0.549	5.900	0.031	0.708
Real GDI (p.c.)	0.343	1.702	-4.075	6.062	0.000	0.912
Unemployment Rate	0.334	5.990	-11.894	31.741	0.000	0.179
Real Rent	0.074	0.867	-3.368	3.881	0.000	0.704
Real Equities	0.239	9.386	-46.671	17.803	0.000	0.653
Real Mortgage Rate*	0.016	3.904	-19.500	15.352	0.000	0.174

Notes: Computed over the entire sample 1972:Q3 - 2017:Q3 period. ADF p-values refer to the Dickey and Fuller (1979) unit-root test of the following hypotheses $H_0 : y_t \sim I(1)$ vs. $H_1 : y_t \sim I(0)$. The optimal lag length in the ADF test equations is selected using the AIC criterion. * computed as Δy_t rather than $\Delta\ln(y_t)$ due to the presence of negative values in the real mortgage rate series.

3.1 Train/Test Set Split

We adopt a fixed estimation window scheme where the algorithms are trained on subsets of the in-sample period via K-fold cross-validation. The selected models are then re-estimated (re-optimized) once on the entire in-sample period, and then used to predict house prices over the out-of-sample time interval.

Figure 2: House Price Index Growth Rates (Log Returns)



After computing growth rates the in-sample period contains 129 quarterly observations and covers the 1972:Q3 - 2004:Q4 time interval. The out-of-sample (test) period is 2005:Q1 - 2017:Q3 consisting of 51 data points. Given the total sample size this results in a 70:30 percent split between in-sample and out-of-sample periods. Figure 2 plots the real house price growth rates over the train and test subsamples. Considering the local minima and maxima we observe that

Electronic copy available at: https://ssrn.com/abstract=3417527

the volatility of the house price growth rates has decreased over the forecast range. This is indeed confirmed by the standard deviation estimates of 2.20 over the in-sample vs. 1.55 across the out-of-sample period. The average of the return series however does not appear to have changed markedly over the two subsamples.

4 Empirical Findings

There are 47 competing forecasts in total, including four simple forecast combinations and a number of ensemble methods, as described in Section 2.1. Table 3 reports out-of-sample forecast analysis of the top 20 forecasts ranked by their relative MSEs, while the remaining predictions are contained in the Appendix. The benchmark model used for computing relative MSEs is the random walk with drift model for log-level prices, or equivalently historical mean prediction for growth rates (Algorithm 2 in Table 1), see Section 2.3 for details. Table 3 also provides p-values from the Diebold and Mariano (1995)/Giacomini and White (2006) (DM/GW) tests of equal unconditional predictive accuracy in square brackets. The last two rows present i) Hochberg-Bonferroni (HB) upper p-value bound (Hochberg, 1988) for testing the multiple hypothesis that all pairwise comparisons are zero, and ii) MSE for the top ranking model of each column representing absolute, rather than relative, forecast accuracy. The algorithms are denoted according to their specification with respect to the house price growth rate series, whose forecasts are integrated into house log price predictions (see Section 2.2.3).

4.1 Forecasting One Quarter Ahead

Prediction errors for 1-quarter ahead forecasts are equal for the house price growth rates and log levels, as demonstrated by the initial two columns of Table 3 and explained in Section 2.2.3. The best performing forecast is generated from a single-equation (autoregressive) model estimated using the support vector regressor with a linear kernel, SVR (Lin)^{se}, and cross-validated with the K-fold method. SVR (Lin)^{se} accuracy is judged by a relatively small forecast error (MSE), which is about 40 percent lower than the MSE of the benchmark model. Moreover, this gain in forecast accuracy is statistically significant at the 5 percent level according to the DM/GW test with a p-value of 0.035. The actual MSE value for SVR (Lin)^{se} is 1.377 which is given in the last row of the table.

The second best model is an ARMA$(2,0)$ whose optimal lag length is chosen by the BIC, rather than cross-validation. It produces a relative MSE of 0.616, and it also outperforms the benchmark at the 5 percent statistical significance according to the DM/GW tests. The top 5 forecast list is completed by vector autoregressions estimated by three different methods. In the third place is a traditional VAR(1) whose lag length is selected by the BIC and which is estimated using the standard quadratic loss function. In contrast, forecasts ranked four and five are generated by VAR models constructed from the VAR(max[p]) template[6], but which are estimated via the Lasso and ElasticNet objective functions, respectively.

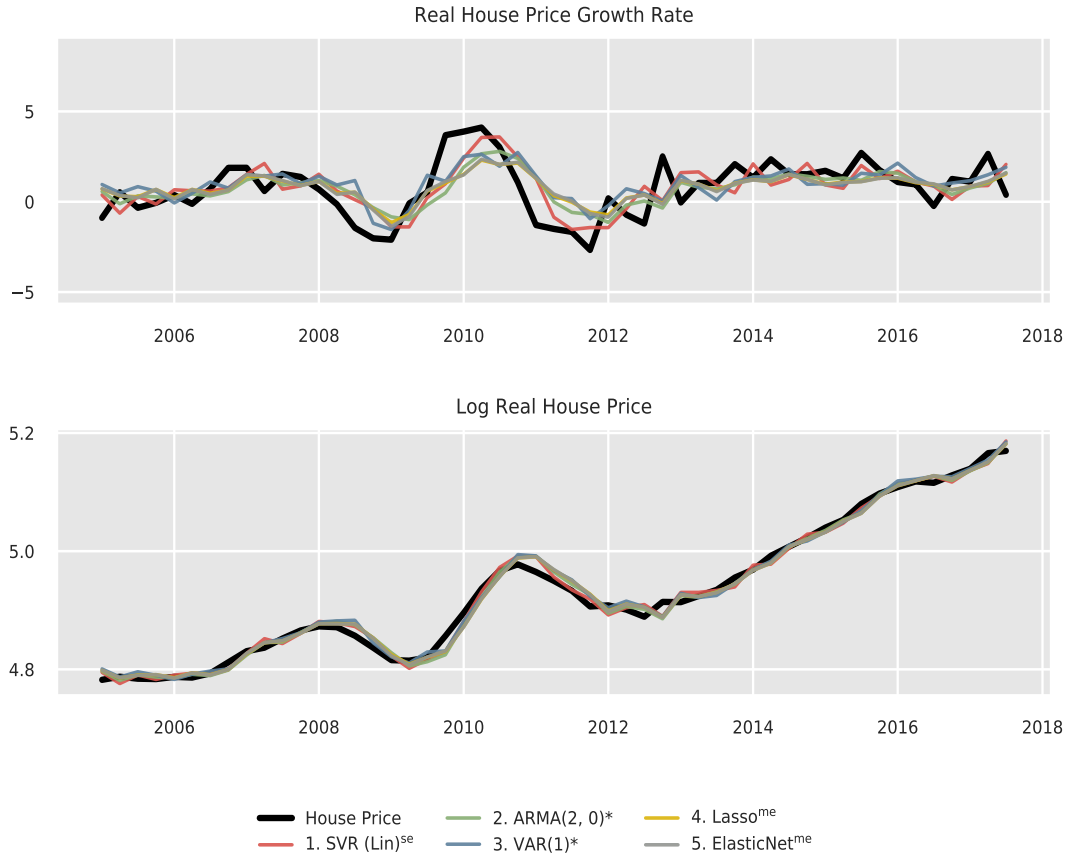
[6]We set the maximum lag length p to 12 quarters throughout this paper, i.e. max[p]=12.

Electronic copy available at: https://ssrn.com/abstract=3417527

The random walk with drift benchmark does not rank in the top 20 forecasts here. In fact the forecast ranking in the 20th place, constructed from a single-equation XGB algorithm, exhibits a MSE of 0.771 relative to the benchmark. It is therefore still about 22.9 percent below the forecast error generated by the random walk with drift model used to forecast log prices. Additionally, all of the top 20 forecasts statistically outperform the benchmark at the 5 percent level, both individually according to the DM/GW tests as well as jointly according to a conservative HB p-value bound.

In relation to linear versus nonlinear specifications we note that the top seven models are all linear specifications. The first nonlinear model which appears on the list is SVR (Poly)^{se} that ranks in the eight place. Of further interest is that both simple forecast combinations feature in the top 20 forecasts, as well as that 16 out of the top 20 forecasts are generated by single-equation models. Given that our multi-equation systems of 8 variables take about 8 times as long to train as single-equation models this has practical implications regarding the amount of computing power and time required to forecast from such systems. Multilayer Perceptron (MLP) ranks in the 15th place, while a more sophisticated LSTM Recurrent Network does not perform as well, being in the 33rd place. Although there are exceptions, most of the 1-quarter ahead forecasts presented in tables A1 and A2 (forecasts ranking in positions 21 – 47) are not statistically different from the benchmarks at the 5 percent level according to both single- and multi-hypothesis tests.

Figure 3: Top Five 1-Quarter Ahead Forecasts (Out-of-Sample)



Electronic copy available at: https://ssrn.com/abstract=3417527

Figure 3 plots the 1-quarter ahead forecasts generated by the top 5 algorithms according to the first two columns of Table 3. It is clear that the top 5 forecasts evolve in a rather close unison over the out-of-sample time period. In the top panel of the figure historical house price growth rate oscillates around the forecasts signifying the difficulty of predicting unexpected economic shocks. Nevertheless, the forecasts appear to be quite good at predicting the general direction of the market. Interestingly SVR $(\text{Lin})^{\text{se}}$, depicted in red, appears to stand out somewhat from the remaining four models in forecasting extreme peaks and troughs. In particular, it provides the closest forecasts to the 2010 peak and the late 2011 trough. Once the five growth rate predictions are integrated into log price forecasts they are virtually indistinguishable from each other, as well as from the house price itself. A notable exception is the overprediction of the log price following its peak in 2011 by all of the top five models.

Electronic copy available at: https://ssrn.com/abstract=3417527

Table 3: Relative MSEs for the Top 20 Out-of-Sample House Price & Growth Rate Predictions

	1 Quarter Ahead Growth Rate	1 Quarter Ahead Log Price	2 Quarters Ahead Growth Rate	2 Quarters Ahead Log Price	4 Quarters Ahead Growth Rate	4 Quarters Ahead Log Price	8 Quarters Ahead Growth Rate	8 Quarters Ahead Log Price
1	SVR (Lin)^{se} 0.581$^{[0.035]}$	SVR (Lin)^{se} 0.581$^{[0.035]}$	SVR (Lin)^{se} 0.713$^{[0.026]}$	SVR (Lin)^{se} 0.553$^{[0.014]}$	Comb. Mean^{me} 0.962$^{[0.230]}$	SVR (Lin)^{se} 0.796$^{[0.063]}$	SVR (Lin)^{se} 0.952$^{[0.126]}$	Comb. Mean^{me} 0.982$^{[0.418]}$
2	ARMA(2, 0)* 0.616$^{[0.011]}$	ARMA(2, 0)* 0.616$^{[0.011]}$	KRR (RBF)^{se} 0.784$^{[0.033]}$	KRR (RBF)^{se} 0.629$^{[0.015]}$	SVR (RBF)^{me} 1.000$^{[0.478]}$	ElasticNet^{se} 0.857$^{[0.121]}$	SVR (Poly)^{se} 0.974$^{[0.201]}$	SVR (RBF)^{me} 0.996$^{[0.431]}$
3	VAR(1)* 0.617$^{[0.018]}$	VAR(1)* 0.617$^{[0.018]}$	ElasticNet^{se} 0.787$^{[0.018]}$	MLP^{se} 0.633$^{[0.013]}$	AR(0)-C 1.000$^{[-]}$	Comb. Mean^{me} 0.864$^{[0.039]}$	KRR (RBF)^{se} 0.984$^{[0.389]}$	KRR (Poly)^{me} 0.999$^{[0.497]}$
4	Lasso^{me} 0.618$^{[0.009]}$	Lasso^{me} 0.618$^{[0.009]}$	MLP^{se} 0.796$^{[0.031]}$	AR(12) 0.635$^{[0.015]}$	SVR (Lin)^{se} 1.005$^{[0.531]}$	KRR (Sig)^{se} 0.869$^{[0.171]}$	DecisionTree^{me} 0.985$^{[0.315]}$	AR(0)-C 1.000$^{[-]}$
5	ElasticNet^{me} 0.621$^{[0.011]}$	ElasticNet^{me} 0.621$^{[0.011]}$	AR(12) 0.810$^{[0.051]}$	ElasticNet^{se} 0.637$^{[0.006]}$	KRR (Sig)^{me} 1.005$^{[0.546]}$	KRR (RBF)^{se} 0.873$^{[0.171]}$	KRR (Sig)^{se} 0.985$^{[0.381]}$	KRR (Sig)^{me} 1.008$^{[0.529]}$
6	ElasticNet^{se} 0.622$^{[0.014]}$	ElasticNet^{se} 0.622$^{[0.014]}$	Lasso^{me} 0.811$^{[0.009]}$	KRR (Sig)^{se} 0.640$^{[0.014]}$	LSTM^{se} 1.010$^{[0.567]}$	AR(12) 0.873$^{[0.178]}$	ElasticNet^{se} 0.985$^{[0.338]}$	KRR (RBF)^{me} 1.046$^{[0.637]}$
7	ARMA(1, 1) 0.628$^{[0.012]}$	ARMA(1, 1) 0.628$^{[0.012]}$	ElasticNet^{me} 0.814$^{[0.014]}$	SVR (Poly)^{se} 0.655$^{[0.007]}$	KRR (Poly)^{me} 1.012$^{[0.586]}$	KRR (Poly)^{se} 0.877$^{[0.163]}$	KRR (Poly)^{se} 0.991$^{[0.429]}$	KNeighbors^{me} 1.067$^{[0.943]}$
8	SVR (Poly)^{se} 0.632$^{[0.014]}$	SVR (Poly)^{se} 0.632$^{[0.014]}$	Comb. Median^{se} 0.817$^{[0.029]}$	KRR (Poly)^{se} 0.658$^{[0.009]}$	LSTM^{me} 1.015$^{[0.628]}$	MLP^{se} 0.891$^{[0.206]}$	SVR (RBF)^{se} 0.997$^{[0.489]}$	Comb. Median^{me} 1.067$^{[0.761]}$
9	Comb. Median^{se} 0.633$^{[0.014]}$	Comb. Median^{se} 0.633$^{[0.014]}$	KRR (Poly)^{se} 0.820$^{[0.042]}$	Lasso^{me} 0.664$^{[0.005]}$	KNeighbors^{me} 1.021$^{[0.813]}$	Comb. Median^{se} 0.899$^{[0.212]}$	MLP^{se} 0.998$^{[0.485]}$	LSTM^{me} 1.070$^{[0.720]}$
10	KRR (Poly)^{se} 0.637$^{[0.018]}$	KRR (Poly)^{se} 0.637$^{[0.018]}$	SVR (Poly)^{se} 0.821$^{[0.045]}$	ElasticNet^{me} 0.664$^{[0.005]}$	ElasticNet^{se} 1.024$^{[0.624]}$	Lasso^{me} 0.899$^{[0.072]}$	AR(0)-C 1.000$^{[-]}$	SVR (Lin)^{se} 1.075$^{[0.646]}$
11	KRR (Sig)^{se} 0.648$^{[0.038]}$	KRR (Sig)^{se} 0.648$^{[0.038]}$	KRR (Sig)^{se} 0.822$^{[0.059]}$	Comb. Median^{se} 0.666$^{[0.008]}$	Comb. Median^{me} 1.024$^{[0.754]}$	ElasticNet^{me} 0.901$^{[0.087]}$	KNeighbors^{se} 1.000$^{[0.500]}$	SVR (Lin)^{me} 1.082$^{[0.645]}$
12	SVR (RBF)^{se} 0.648$^{[0.019]}$	SVR (RBF)^{se} 0.648$^{[0.019]}$	Comb. Mean^{se} 0.827$^{[0.045]}$	SVR (RBF)^{se} 0.673$^{[0.015]}$	SVR (Lin)^{me} 1.027$^{[0.625]}$	Comb. Mean^{se} 0.911$^{[0.245]}$	XGB^{se} 1.000$^{[0.502]}$	KRR (Poly)^{se} 1.082$^{[0.665]}$
13	KRR (RBF)^{se} 0.650$^{[0.043]}$	KRR (RBF)^{se} 0.650$^{[0.043]}$	LGBM^{me} 0.829$^{[0.009]}$	ARMA(2, 0)* 0.681$^{[0.007]}$	LGBM^{me} 1.029$^{[0.796]}$	KRR (Poly)^{me} 0.923$^{[0.139]}$	SVR (RBF)^{me} 1.001$^{[0.536]}$	ElasticNet^{se} 1.089$^{[0.678]}$
14	AR(12) 0.651$^{[0.044]}$	AR(12) 0.651$^{[0.044]}$	SVR (RBF)^{se} 0.832$^{[0.074]}$	Comb. Mean^{se} 0.689$^{[0.014]}$	KRR (RBF)^{me} 1.033$^{[0.694]}$	SVR (Poly)^{se} 0.928$^{[0.306]}$	Comb. Mean^{se} 1.003$^{[0.529]}$	LSTM^{se} 1.100$^{[0.842]}$
15	MLP^{se} 0.653$^{[0.045]}$	MLP^{se} 0.653$^{[0.045]}$	ARMA(2, 0)* 0.839$^{[0.022]}$	VAR(1)* 0.689$^{[0.025]}$	VAR(1)* 1.035$^{[0.686]}$	VAR(1)* 0.935$^{[0.256]}$	Lasso^{se} 1.004$^{[0.772]}$	SVR (Poly)^{me} 1.126$^{[0.795]}$
16	Lasso^{se} 0.659$^{[0.007]}$	Lasso^{se} 0.659$^{[0.007]}$	VAR(1)* 0.852$^{[0.092]}$	ARMA(1, 1) 0.705$^{[0.005]}$	SVR (Poly)^{me} 1.046$^{[0.757]}$	LGBM^{me} 0.937$^{[0.180]}$	LGBM^{me} 1.006$^{[0.598]}$	KRR (Sig)^{se} 1.132$^{[0.718]}$
17	Comb. Mean^{se} 0.662$^{[0.018]}$	Comb. Mean^{se} 0.662$^{[0.018]}$	Lasso^{se} 0.853$^{[0.003]}$	Lasso^{se} 0.717$^{[0.003]}$	Lasso^{me} 1.052$^{[0.827]}$	Comb. Median^{me} 0.942$^{[0.137]}$	LSTM^{me} 1.008$^{[0.566]}$	MLP^{se} 1.133$^{[0.733]}$
18	SVR (Sig)^{se} 0.702$^{[0.030]}$	SVR (Sig)^{se} 0.702$^{[0.030]}$	ARMA(1, 1) 0.874$^{[0.033]}$	LGBM^{me} 0.753$^{[0.003]}$	ElasticNet^{me} 1.054$^{[0.808]}$	Lasso^{se} 0.950$^{[0.234]}$	AR(12) 1.009$^{[0.589]}$	LGBM^{me} 1.135$^{[0.834]}$
19	Comb. Mean^{me} 0.745$^{[0.032]}$	Comb. Mean^{me} 0.745$^{[0.032]}$	Comb. Mean^{me} 0.880$^{[0.068]}$	SVR (Sig)^{se} 0.757$^{[0.049]}$	Lasso^{se} 1.057$^{[0.921]}$	KRR (RBF)^{me} 0.968$^{[0.351]}$	Comb. Median^{se} 1.010$^{[0.622]}$	GradientBoost^{se} 1.138$^{[0.762]}$
20	XGB^{se} 0.747$^{[0.030]}$	XGB^{se} 0.747$^{[0.030]}$	LGBM^{se} 0.889$^{[0.187]}$	Comb. Mean^{me} 0.771$^{[0.021]}$	KRR (Poly)^{se} 1.063$^{[0.800]}$	SVR (RBF)^{se} 0.977$^{[0.444]}$	KRR (Sig)^{me} 1.011$^{[0.597]}$	KRR (RBF)^{se} 1.139$^{[0.731]}$
HB (p-value)	[0.040]	[0.040]	[0.060]	[0.040]	[0.920]	[0.440]	[0.770]	[0.940]
MSE (Best Model)	1.377	1.377	1.682	4.324	2.335	19.078	2.479	39.578

Notes: Rankings according to the MSEs relative to the benchmark random walk with drift model for log prices (historical average for growth rates). Square brackets contain p-values from DM/GW tests of equal predictive accuracy. MSE(Best Model)=MSE from the top-ranking model; se=single-equation; me=multi-equation; *=lag length chosen by the BIC; HB=Hochberg-Bonferroni upper p-value bound for testing that all pairwise comparisons are zero; Maximum lag length=12 quarters.

Electronic copy available at: https://ssrn.com/abstract=3417527

4.2 Forecasting Two Quarters Ahead

Relative MSEs computed for the 2-step ahead forecasts are presented across columns three and four of tables 3, and A1, A2. As in the case of 1-quarter ahead forecast discussed above, support vector regressor SVR (Lin)^{se} ranks first in forecasting both the growth rate as well as the log house price. It exhibits a relative MSE of 0.713 in forecasting the growth rate, thus outperforming the benchmark model by almost 30 percent. SVR (Lin)^{se} is, however, even more impressive in forecasting real log prices with the relative MSE of 0.553. Both of these outpredictions are statistically significant at the 5 percent level according to the DM/GW tests of equal predictive accuracy. SVR (Lin)^{se} produces (absolute) MSEs for the growth rate and log prices series of 1.682 and 4.324, as listed in the last row of the table. These are greater than their corresponding values for the 1-quarter ahead forecasts, reflecting greater uncertainty associated with making predictions over longer forecast horizons.

The second place is taken by KRR (RBF)^{se} for both log prices and growth rates. The rest of the algorithms typically do not rank equally between house price growth rates and log levels, although certain models perform well in forecasting both of these series. For instance, the top 5 list for the log price is completed by MLP^{se}, AR(12) and ElasticNet^{se}, while the best 5 growth rates forecasts are generated by these same algorithms although with permuted rankings. Interestingly, all top five 2-step ahead forecasts are generated from univariate (single-equation) rather than multivariate (multi-equation) models, while two of the top 5 forecasts are generated by nonlinear specifications.

The random walk with drift benchmark model does not rank in the top 20 forecasts. In fact, the forecasts of the growth rate and log price ranked last in Table 3 still outperform the benchmark by about 10 and 20 percent, respectively. The outperformance by most of the top 20 algorithms against the benchmark is confirmed by the DM/GW tests at the 5 percent level. The conservative HB p-value bound for testing that all of the top 20 models outperform the benchmark jointly is 0.04 in the case of log house prices and 0.06 for the growth rates. This provides strong support in favour of top 20 specifications over the benchmark. According to Table A1 the benchmarks themselves rank in positions 30 and 33 for the growth rates and log prices, respectively.

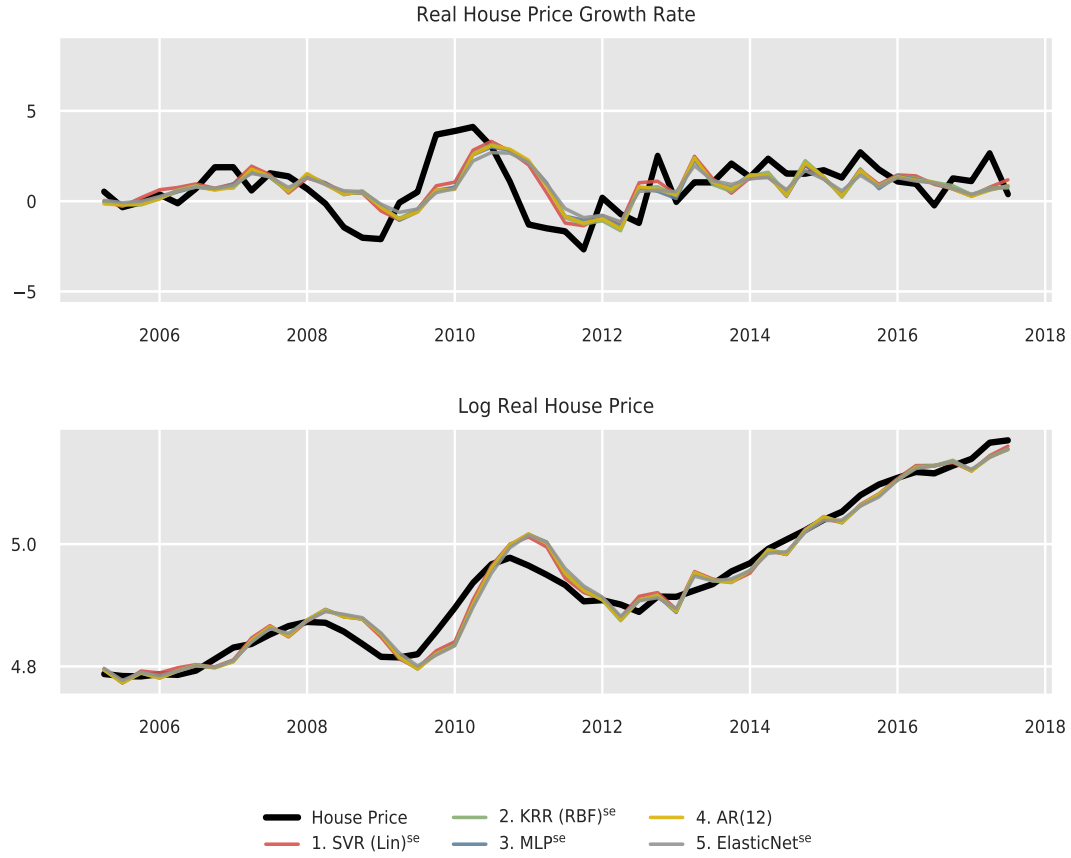
Fifteen out of the top 20 forecasts for both the log prices and growth rates are univariate rather than multi-variate specifications. This provides evidence in favour of simpler models, which are also less demanding in terms of training time and computational power. Regarding the suitability of linear models to produce adequate forecasts we find that 10 out of the top 20 forecasts are linear for both growth rates and log prices. Thus, while there is scope for using nonlinear specifications the evidence is not overwhelmingly in favour of more complicated conditional mean functions. Lastly, deep learning multilayer perceptron MLP^{se} ranks in the 3rd and 4th places for the log price and growth rates, respectively.

Figure 4 depicts the top 5 algorithms chosen for predicting log house prices[7] two quarters ahead. Similar to the case of 1 quarter forecast horizon, all top 5 forecasts are closely bunched

[7]These are in fact also the same top 5 algorithms for growth rates according to Table 3.

Electronic copy available at: https://ssrn.com/abstract=3417527

Figure 4: Top Five 2-Quarter Ahead Forecasts (Out-of-Sample)



together. The forecasts now lag the two house price series more than was the case in Figure 3, especially in regards to forecasting the 2010 peak. The algorithms seem to underpredict the exuberant growth rates recorded over the 2013 – 2016 period. This is likely due to the fact that the model is trained only once using a fixed-estimation window scheme. Nevertheless, when the growth forecast are integrated into the log price forecasts they appear to do quite well since 2013 as illustrated in the bottom panel of Figure 4.

4.3 Forecasting Four and Eight Quarters Ahead

Accuracy of the house price growth rate and log level predictions over 4- and 8-quarter forecast horizons is markedly different from previously discussed shorter term forecasts. We formulate our discussion by first considering the growth rate forecasts presented in columns 5 and 7 of tables 3, and A1, A2. This is followed by our observations regarding the log price forecasts provided in columns 6 and 8 of the same tables.

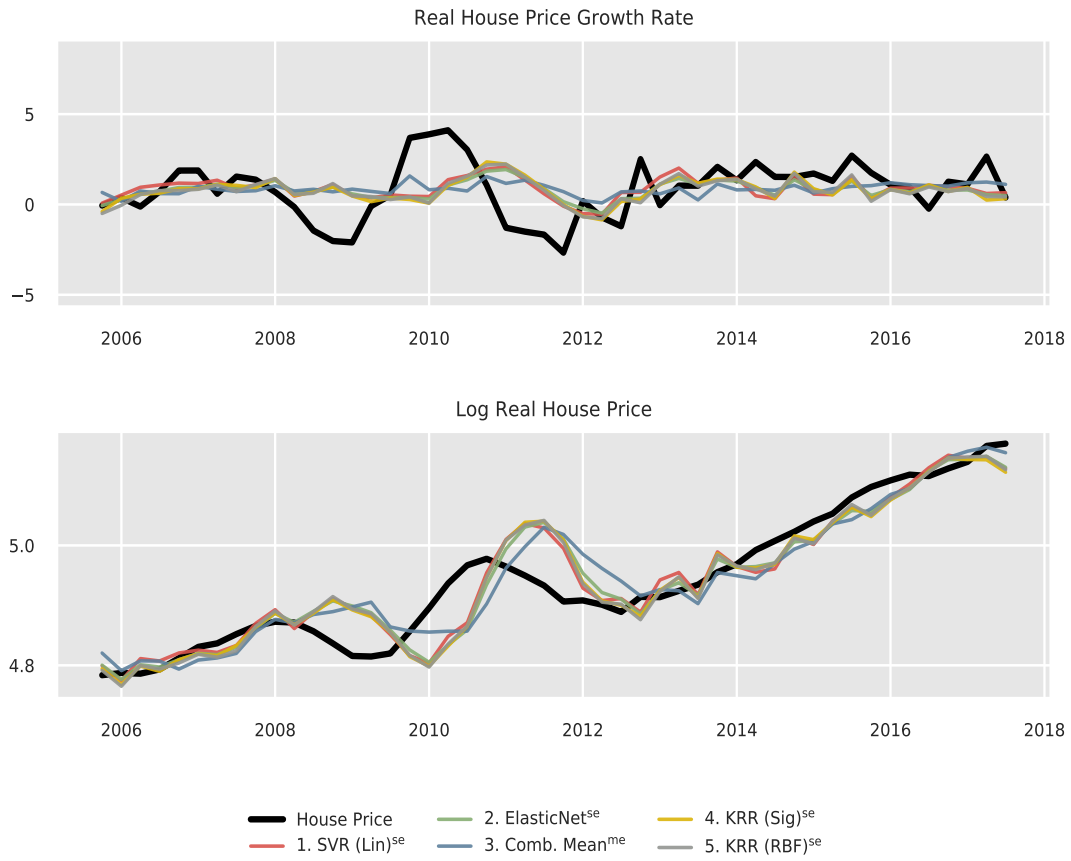
When forecasting house price growth rates 4-quarters ahead there is only one algorithm – simple mean combination of multivariate models – that outperforms the benchmark of historical mean prediction. The benchmark itself shares the 2nd palace with the support vector regressor SVR (RBF)^{me}. Considering that the relative MSE of the top model is 0.962 its outperformance seems quite marginal at less than 4 percent. In the case of forecasts made for 8 quarters ahead

Electronic copy available at: https://ssrn.com/abstract=3417527

there are 9 models that outperform the benchmark. However their outperformance is also small ranging from about 5 percent to 0.2 percent. None of the outpredictions of the benchmark discussed here are statistically significant at the 10 percent level according to the DM/GW p-values given in square brackets. In fact, not one of the 47 models presented in columns 5 and 7 of tables 3, and A1, A2 outperforms the benchmark at the 10 percent level.

Considering the 4-quarter ahead forecasts of the log real house price provided in column 6 of Table 3 we make the following observations. First, the relative MSEs are generally smaller than their counterparts for forecasting growth rates (column 5). Thus it appears that there is greater value in employing more complicated methods when forecasting the log price itself. The first place is taken by SVR (Lin)^{se} with a relative MSE of 0.796. This outperformance appears to be statistically significant at the 10 percent level with the p-value of 0.063. A number of other algorithms, such as the mean forecast combination ranked number 3, also statistically outperform the benchmark. Lastly, the HB p-value bound is 0.440. This suggests that we accept the null of equal performance accuracy of each alternative model and the benchmark, jointly.

Figure 5: Top Five 4-Quarter Ahead Forecasts (Out-of-Sample)



Forecasting the log price 8 quarters ahead produces similar results. The benchmark random walk with drift now ranks in the 4th position. As illustrated in the last column of tables 3, A1, A2 a large majority of the algorithms produce substantially larger MSEs than the benchmark.

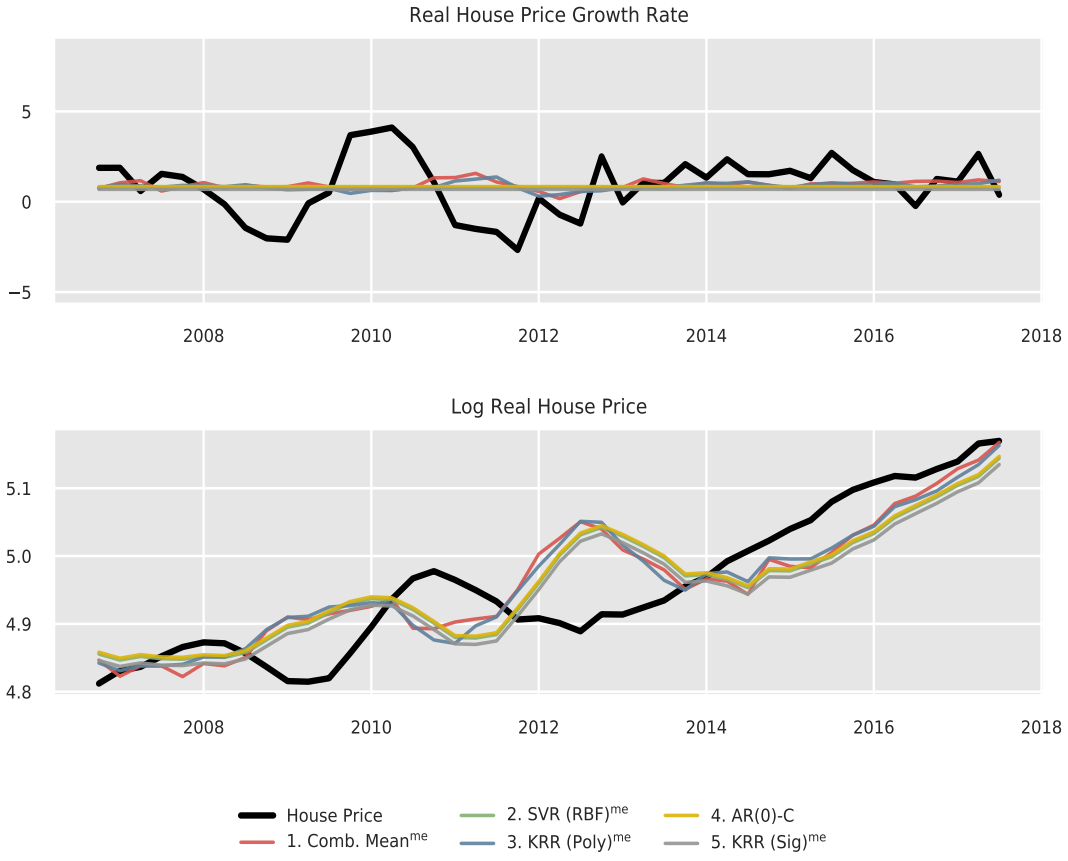
Electronic copy available at: https://ssrn.com/abstract=3417527

The HB p-value bounds from multiple hypothesis of equal predictive accuracies are in access of 0.94 across the three tables suggesting that it is difficult to outperform the benchmark model when forecasting 8 quarters ahead.

While the above paragraphs discuss outperformance relative to the benchmark, the last row of Table 3 provides actual MSEs for the top performing models. As expected, forecasting over longer horizons entails more risk as compared to shorter horizon predictions. However, there is a striking pattern in the MSEs, where the largest increase in forecast uncertainty happens between 2 and 4 quarter forecast horizons. This is best illustrated by more than a four-fold increase in the MSE for the log price predictions (from 4.324 to 19.078). By comparison, the corresponding MSE increases from 1 to 2, and from 4 to 8 quarter horizons are much smaller in magnitudes.

Figures 5 and 6 depict top 5 algorithms selected according to the relative MSEs for predicting log house prices with 4- and 8-quarter forecast horizons, respectively. Regarding the 4-quarter ahead forecasts, we observe a tight grouping of the predictions for both the growth rates as well as for log prices. The predictions appear adequate for the pre-2008 and post-2012 periods. In contrast, the period of the global financial crisis around 2008 and the subsequent recovery is quite difficult to predict. Considering the bottom panel of Figure 5 we see that the 2008 house price peak, the late 2009 trough and the subsequent 2010 peak were all predicted with substantial delays.

Figure 6: Top Five 8-Quarter Ahead Forecasts (Out-of-Sample)



Electronic copy available at: https://ssrn.com/abstract=3417527

Top panel of Figure 6 illustrates a well known feature of stationary time series stating that predictions converge to the unconditional mean of the series as the forecast horizon grows larger. It seems that the horizon of 8 quarters is sufficiently large for this result to come into effect here. Although the pre-2008 period is forecast well, the 8-quarter ahead predictions completely miss the late 2009 trough, the 2010 peak and the subsequent 2012 trough. In the post-2014 period we see a convergence of the forecasts towards the realised log house prices.

5 Conclusions

We compare forecasting ability of 47 alternative algorithms to produce accurate out-of-sample forecasts of the Australian real house log levels and growth rates. The predictions are generated from a range of traditional time series models, e.g. ARMA and VAR, machine learning algorithms such as kernel ridge regressions, as well as deep learning neural networks. A large majority of the algorithms are specified in two forms: i) as univariate (single-equation) models, and as ii) multivariate systems of equations containing additional predictive variables. All models are estimated only once on the 1972:Q4 – 2004:Q4 in-sample period. Iterated forecasts are then computed for 1, 2, 4, and 8 quarters ahead over the 2005:Q1 – 2017:Q3 out-of-sample period. We generate iterated predictions from nonlinear algorithms using a method adopted from Brown and Mariano (1989). Predictive accuracy is judged according to MSEs relative to the random walk with drift benchmark for log prices, and equivalently historical average prediction for house price growth rates. Additionally, formal tests of equal predictive accuracy are performed.

We attempt to summarise our findings across 8 forecast horizon/dependent variable configurations (4 forecast horizons $\times$ 2 dependent variables). First, best forecasts appear to be generated by a univariate linear support vector regressor which ranks first in 6 out of the 8 forecast-horizon/dependent-variable combinations. This method is found to reduce the forecast error by up to 45 percent relative to the random walk with drift benchmark. Second, comparing single-equation (univariate) and multi-equation (multivariate) specifications we report that univariate models prevail in the top 10 predictions across most forecast horizons. This has significant implications regarding time and computing power requirements needed to train and optimise good forecasting models. Third, an interesting pattern emerges in relation to the linear versus nonlinear conditional mean specifications. Linear models appear more suited for making 1-quarter ahead predictions, while the results are not clear cut regarding 2- and 4-quarter horizons. In contrast, nonlinear specifications dominate the top 10 predictions when forecasting 8 quarters ahead. Lastly, both absolute and relative prediction accuracy worsen with the length of the forecast horizon. Thus, while we find substantial gains from constructing predictive models for shorter forecast horizons of 1 and 2 quarters, and some benefits for 4-quarter ahead predictions, longer-term forecasts of 8 quarters may simply rely on the random walk with drift benchmark.

Electronic copy available at: https://ssrn.com/abstract=3417527

References

Abelson, P., Joyeux, R., Milunovich, G. and Chung, D. (2005), 'Explaining house prices in australia: 1970–2003', *Economic Record* **81**, 96–103.

Abraham, J. M. and Hendershott, P. H. (1996), 'Bubbles in metropolitan housing markets', *Journal of Housing Research* p. 191–207.

Balcilar, M., Gupta, R. and Miller, S. M. (2015), 'The out-of-sample forecasting performance of nonlinear models of regional housing prices in the us', *Applied Economics* **47**(22), 2259–2277.

Bergmeir, C., Hyndman, R. J. and Koo, B. (2018), 'A note on the validity of cross-validation for evaluating autoregressive time series prediction', *Computational Statistics & Data Analysis* **120**, 70–83.

Bork, L. and Møller, S. V. (2015), 'Forecasting house prices in the 50 states using dynamic model averaging and dynamic model selection', *International Journal of Forecasting* **31**(1), 63–78.

Bork, L. and Møller, S. V. (2018), 'Housing price forecastability: A factor analysis', *Real Estate Economics* **46**(3), 582–611.

Bourassa, S. C. and Hendershott, P. H. (1995), 'Australian capital city real house prices, 1979–1993', *Australian Economic Review* **28**(3), 16–26.

Breiman, L. (2001), 'Random forests', *Machine learning* **45**(1), 5–32.

Brown, B. W. and Mariano, R. S. (1984), 'Residual-based procedures for prediction and estimation in a nonlinear simultaneous system', *Econometrica: Journal of the Econometric Society* p. 321–343.

Brown, B. W. and Mariano, R. S. (1989), 'Predictors in dynamic nonlinear models: large-sample behavior', *Econometric Theory* **5**(3), 430–452.

Case, K. E. and Shiller, R. J. (1989), 'The efficiency of the market for single-family homes', *The American Economic Review* p. 125–137.

Case, K. E. and Shiller, R. J. (1990), 'Forecasting prices and excess returns in the housing market', *Real Estate Economics* **18**(3), 253–273.

Chen, T. and Guestrin, C. (2016), Xgboost: A scalable tree boosting system, *in* 'Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining', ACM, p. 785–794.

Clark, T. E. and McCracken, M. W. (2001), 'Tests of equal forecast accuracy and encompassing for nested models', *Journal of econometrics* **105**(1), 85–110.

Clark, T. E. and West, K. D. (2007), 'Approximately normal tests for equal predictive accuracy in nested models', *Journal of econometrics* **138**(1), 291–311.

Electronic copy available at: https://ssrn.com/abstract=3417527

Crawford, G. W. and Fratantoni, M. C. (2003), 'Assessing the forecasting performance of regime-switching, arima and garch models of house prices', *Real Estate Economics* **31**(2), 223–243.

Cybenko, G. (1989), 'Approximation by superpositions of a sigmoidal function', *Mathematics of control, signals and systems* **2**(4), 303–314.

Dickey, D. A. and Fuller, W. A. (1979), 'Distribution of the estimators for autoregressive time series with a unit root', *Journal of the American statistical association* **74**(366a), 427–431.

Diebold, F. X. and Mariano, R. S. (1995), 'Comparing predictive accuracy', *Journal of Business & Economic Statistics* p. 253–263.

DiPasquale, D. and Wheaton, W. C. (1994), 'Housing market dynamics and the future of housing prices', *Journal of Urban Economics* **35**(1), 1–27.

Elliott, G. and Timmermann, A. (2016), *Economic Forecasting*, 1 edn, Princeton University Press.

Engelhardt, G. V. (2003), 'Nominal loss aversion, housing equity constraints, and household mobility: evidence from the united states', *Journal of urban Economics* **53**(1), 171–195.

Favilukis, J., Ludvigson, S. C. and Van Nieuwerburgh, S. (2017), 'The macroeconomic effects of housing wealth, housing finance, and limited risk sharing in general equilibrium', *Journal of Political Economy* **125**(1), 140–223.

Friedman, J. H. (2001), 'Greedy function approximation: a gradient boosting machine', *Annals of statistics* p. 1189–1232.

Gau, G. W. (1984), 'Weak form tests of the efficiency of real estate investment markets', *Financial Review* **19**(4), 301–320.

Gau, G. W. (1985), 'Public information and abnormal returns in real estate investment', *Real Estate Economics* **13**(1), 15–31.

Genesove, D. and Mayer, C. (2001), 'Loss aversion and seller behavior: Evidence from the housing market', *The Quarterly Journal of Economics* **116**(4), 1233–1260.

Ghysels, E., Plazzi, A., Valkanov, R. and Torous, W. (2013), Forecasting real estate prices, *in* 'Handbook of Economic Forecasting', Vol. 2, Elsevier, p. 509–580.

Giacomini, R. and White, H. (2006), 'Tests of conditional predictive ability', *Econometrica* **74**(6), 1545–1578.

Gupta, R., Kabundi, A. and Miller, S. M. (2011), 'Forecasting the us real house price index: Structural and non-structural models with and without fundamentals', *Economic Modelling* **28**(4), 2013–2021.

Electronic copy available at: https://ssrn.com/abstract=3417527

Hastie, T., Tibshirani, R. and Friedman, J. (2009), 'The elements of statistical learning: data mining, inference, and prediction, springer series in statistics'.

Hochberg, Y. (1988), 'A sharper bonferroni procedure for multiple tests of significance', *Biometrika* **75**(4), 800–802.

Hochreiter, S. and Schmidhuber, J. (1997), 'Long short-term memory', *Neural computation* **9**(8), 1735–1780.

Hoerl, A. E. and Kennard, R. W. (1970), 'Ridge regression: applications to nonorthogonal problems', *Technometrics* **12**(1), 69–82.

Hornik, K., Stinchcombe, M. and White, H. (1989), 'Multilayer feedforward networks are universal approximators', *Neural networks* **2**(5), 359–366.

Malpezzi, S. (1999), 'A simple error correction model of house prices', *Journal of Housing Economics* **8**(1), 27–62.

McIntosh, W. and Henderson, G. V. (1989), 'Efficiency of the office properties market', *The Journal of Real Estate Finance and Economics* **2**(1), 61–70.

Miles, W. (2008), 'Boom–bust cycles and the forecasting performance of linear and non-linear models of house prices', *The Journal of Real Estate Finance and Economics* **36**(3), 249–264.

Muellbauer, J. and Murphy, A. (1997), 'Booms and busts in the uk housing market', *The Economic Journal* **107**(445), 1701–1727.

Plakandaras, V., Gupta, R., Gogas, P. and Papadimitriou, T. (2015), 'Forecasting the us real house price index', *Economic Modelling* **45**, 259–267.

Plazzi, A., Torous, W. and Valkanov, R. (2010), 'Expected returns and expected growth in rents of commercial real estate', *The Review of Financial Studies* **23**(9), 3469–3519.

Rapach, D. E., Strauss, J. K. et al. (2007), 'Forecasting real housing price growth in the eighth district states', *Federal Reserve Bank of St. Louis. Regional Economic Development* **3**(2), 33–42.

Schindler, F. (2013), 'Predictability and persistence of the price movements of the s&p/case-shiller house price indices', *The Journal of Real Estate Finance and Economics* **46**(1), 44–90.

Schwarz, G. (1978), 'Estimating the dimension of a model', *The annals of statistics* **6**(2), 461–464.

Tibshirani, R. (1996), 'Regression shrinkage and selection via the lasso', *Journal of the Royal Statistical Society: Series B (Methodological)* **58**(1), 267–288.

West, K. D. (1996), 'Asymptotic inference about predictive ability', *Econometrica: Journal of the Econometric Society* p. 1067–1084.

Electronic copy available at: https://ssrn.com/abstract=3417527

Zou, H. and Hastie, T. (2005), 'Regularization and variable selection via the elastic net', *Journal of the royal statistical society: series B (statistical methodology)* **67**(2), 301–320.

6 Appendix

Electronic copy available at: https://ssrn.com/abstract=3417527

Table A1: Remaining Out-of-Sample House Price Predictions - Part 1

	1 Quarter Ahead Growth Rate	1 Quarter Ahead Log Price	2 Quarters Ahead Growth Rate	2 Quarters Ahead Log Price	4 Quarters Ahead Growth Rate	4 Quarters Ahead Log Price	8 Quarters Ahead Growth Rate	8 Quarters Ahead Log Price
21	LGBM^{me} 0.769$^{[0.010]}$	LGBM^{me} 0.769$^{[0.010]}$	Comb. Median^{me} 0.907$^{[0.039]}$	LGBM^{se} 0.800$^{[0.099]}$	Comb. Mean^{se} 1.068$^{[0.779]}$	XGB^{me} 0.991$^{[0.462]}$	Lasso^{me} 1.011$^{[0.954]}$	AR(12) 1.147$^{[0.735]}$
22	XGB^{me} 0.793$^{[0.050]}$	XGB^{me} 0.793$^{[0.050]}$	XGB^{se} 0.912$^{[0.162]}$	XGB^{se} 0.804$^{[0.041]}$	GradientBoost^{me} 1.069$^{[0.935]}$	LSTM^{se} 0.993$^{[0.463]}$	KNeighbors^{me} 1.013$^{[0.861]}$	Comb. Median^{se} 1.161$^{[0.789]}$
23	LGBM^{se} 0.803$^{[0.096]}$	LGBM^{se} 0.803$^{[0.096]}$	KRR (Poly)^{me} 0.914$^{[0.149]}$	Comb. Median^{me} 0.819$^{[0.008]}$	XGB^{me} 1.070$^{[0.847]}$	SVR (RBF)^{me} 0.997$^{[0.420]}$	ElasticNet^{me} 1.014$^{[0.895]}$	Lasso^{se} 1.175$^{[0.897]}$
24	GradientBoost^{me} 0.803$^{[0.091]}$	GradientBoost^{me} 0.803$^{[0.091]}$	XGB^{me} 0.914$^{[0.107]}$	XGB^{me} 0.827$^{[0.035]}$	KRR (RBF)^{se} 1.070$^{[0.782]}$	KRR (Sig)^{me} 0.999$^{[0.495]}$	SVR (Poly)^{me} 1.021$^{[0.661]}$	Comb. Mean^{se} 1.179$^{[0.825]}$
25	Comb. Median^{me} 0.806$^{[0.014]}$	Comb. Median^{me} 0.806$^{[0.014]}$	SVR (Sig)^{se} 0.925$^{[0.299]}$	KRR (Poly)^{me} 0.835$^{[0.041]}$	Comb. Median^{se} 1.078$^{[0.803]}$	LGBM^{se} 1.000$^{[0.499]}$	Comb. Median^{me} 1.027$^{[0.841]}$	Lasso^{me} 1.180$^{[0.881]}$
26	KRR (Poly)^{me} 0.822$^{[0.035]}$	KRR (Poly)^{me} 0.822$^{[0.035]}$	KRR (RBF)^{me} 0.933$^{[0.200]}$	GradientBoost^{me} 0.850$^{[0.097]}$	KRR (Sig)^{se} 1.078$^{[0.764]}$	AR(0)-C 1.000$^{[-]}$	ARMA(1, 1) 1.028$^{[0.963]}$	XGB^{me} 1.187$^{[0.822]}$
27	KRR (RBF)^{me} 0.853$^{[0.054]}$	KRR (RBF)^{me} 0.853$^{[0.054]}$	GradientBoost^{me} 0.942$^{[0.261]}$	KRR (RBF)^{me} 0.870$^{[0.075]}$	XGB^{se} 1.079$^{[0.856]}$	XGB^{se} 1.008$^{[0.526]}$	ARMA(2, 0)* 1.032$^{[0.953]}$	ElasticNet^{me} 1.189$^{[0.878]}$
28	SVR (Sig)^{me} 0.856$^{[0.235]}$	SVR (Sig)^{me} 0.856$^{[0.235]}$	LSTM^{se} 0.995$^{[0.479]}$	SVR (Sig)^{me} 0.903$^{[0.314]}$	AR(12) 1.084$^{[0.806]}$	GradientBoost^{me} 1.011$^{[0.551]}$	XGB^{me} 1.032$^{[0.786]}$	DecisionTree^{me} 1.213$^{[0.988]}$
29	RandomForest^{se} 0.873$^{[0.198]}$	RandomForest^{se} 0.873$^{[0.198]}$	SVR (RBF)^{me} 0.999$^{[0.444]}$	LSTM^{se} 0.908$^{[0.154]}$	DecisionTree^{me} 1.103$^{[0.890]}$	ARMA(2, 0)* 1.015$^{[0.545]}$	KRR (RBF)^{me} 1.036$^{[0.757]}$	XGB^{se} 1.222$^{[0.920]}$
30	GradientBoost^{se} 0.893$^{[0.168]}$	GradientBoost^{se} 0.893$^{[0.168]}$	AR(0)-C 1.000$^{[-]}$	GradientBoost^{se} 0.937$^{[0.260]}$	LGBM^{se} 1.118$^{[0.830]}$	ARMA(1, 1) 1.025$^{[0.582]}$	SVR (Lin)^{me} 1.037$^{[0.668]}$	GradientBoost^{me} 1.231$^{[0.951]}$
31	RandomForest^{me} 0.906$^{[0.280]}$	RandomForest^{me} 0.906$^{[0.280]}$	KRR (Sig)^{me} 1.002$^{[0.512]}$	RandomForest^{se} 0.981$^{[0.449]}$	GradientBoost^{se} 1.124$^{[0.858]}$	SVR (Lin)^{me} 1.026$^{[0.585]}$	VAR(1)* 1.040$^{[0.751]}$	LGBM^{se} 1.237$^{[0.838]}$
32	Ridge^{se} 0.930$^{[0.386]}$	Ridge^{se} 0.930$^{[0.386]}$	KNeighbors^{me} 1.009$^{[0.632]}$	SVR (RBF)^{me} 0.998$^{[0.428]}$	SVR (Poly)^{se} 1.135$^{[0.877]}$	SVR (Sig)^{se} 1.030$^{[0.555]}$	LSTM^{se} 1.050$^{[0.905]}$	SVR (Poly)^{se} 1.260$^{[0.847]}$
33	LSTM^{se} 0.968$^{[0.384]}$	LSTM^{se} 0.968$^{[0.384]}$	LSTM^{me} 1.019$^{[0.660]}$	KRR (Sig)^{me} 0.999$^{[0.497]}$	MLP^{se} 1.138$^{[0.900]}$	KNeighbors^{me} 1.031$^{[0.858]}$	KRR (Poly)^{me} 1.050$^{[0.782]}$	VAR(1)* 1.299$^{[0.890]}$
34	KRR (Sig)^{me} 0.997$^{[0.472]}$	KRR (Sig)^{me} 0.997$^{[0.472]}$	SVR (Lin)^{me} 1.020$^{[0.595]}$	AR(0)-C 1.000$^{[-]}$	KNeighbors^{se} 1.146$^{[0.976]}$	LSTM^{me} 1.033$^{[0.694]}$	GradientBoost^{me} 1.053$^{[0.903]}$	KNeighbors^{se} 1.396$^{[0.956]}$
35	SVR (RBF)^{me} 0.997$^{[0.406]}$	SVR (RBF)^{me} 0.997$^{[0.406]}$	SVR (Poly)^{me} 1.027$^{[0.657]}$	KNeighbors^{me} 1.013$^{[0.662]}$	SVR (RBF)^{se} 1.160$^{[0.898]}$	SVR (Poly)^{me} 1.061$^{[0.748]}$	Comb. Mean^{me} 1.061$^{[0.807]}$	ARMA(1, 1) 1.442$^{[0.945]}$
36	SVR (Poly)^{me} 1.000$^{[0.498]}$	SVR (Poly)^{me} 1.000$^{[0.498]}$	GradientBoost^{se} 1.029$^{[0.605]}$	SVR (Poly)^{me} 1.016$^{[0.585]}$	SVR (Sig)^{se} 1.179$^{[0.794]}$	GradientBoost^{se} 1.065$^{[0.679]}$	DecisionTree^{se} 1.076$^{[0.981]}$	ARMA(2, 0)* 1.454$^{[0.950]}$
37	AR(0)-C 1.000$^{[-]}$	AR(0)-C 1.000$^{[-]}$	DecisionTree^{me} 1.031$^{[0.719]}$	SVR (Lin)^{me} 1.020$^{[0.581]}$	ARMA(1, 1) 1.183$^{[0.948]}$	DecisionTree^{me} 1.119$^{[0.932]}$	LGBM^{se} 1.083$^{[0.778]}$	SVR (Sig)^{se} 1.471$^{[0.878]}$
38	KNeighbors^{se} 1.006$^{[0.534]}$	KNeighbors^{se} 1.006$^{[0.534]}$	SVR (Sig)^{me} 1.061$^{[0.627]}$	LSTM^{me} 1.025$^{[0.677]}$	ARMA(2, 0)* 1.184$^{[0.945]}$	SVR (Sig)^{me} 1.141$^{[0.737]}$	GradientBoost^{se} 1.089$^{[0.892]}$	SVR (RBF)^{se} 1.473$^{[0.933]}$
39	SVR (Lin)^{me} 1.012$^{[0.557]}$	SVR (Lin)^{me} 1.012$^{[0.557]}$	KNeighbors^{se} 1.063$^{[0.792]}$	KNeighbors^{se} 1.037$^{[0.698]}$	DecisionTree^{se} 1.188$^{[0.920]}$	KNeighbors^{se} 1.194$^{[0.947]}$	RandomForest^{se} 1.097$^{[0.986]}$	DecisionTree^{se} 1.511$^{[0.952]}$
40	KNeighbors^{me} 1.020$^{[0.729]}$	KNeighbors^{me} 1.020$^{[0.729]}$	RandomForest^{se} 1.094$^{[0.740]}$	DecisionTree^{se} 1.040$^{[0.604]}$	SVR (Sig)^{me} 1.231$^{[0.891]}$	DecisionTree^{se} 1.217$^{[0.892]}$	SVR (Sig)^{se} 1.102$^{[0.835]}$	SVR (Sig)^{me} 1.550$^{[0.908]}$
HB (p-value)	[0.190]	[0.190]	[0.740]	[0.150]	[0.970]	[0.940]	[0.980]	[0.980]
MSE (Best Model)	1.824	1.824	2.139	6.261	2.591	23.771	2.631	46.233

Notes: Rankings according to MSEs relative to the benchmark random walk with drift model for log prices (historical average for growth rates). Square brackets contain p-values from DM/GW tests of equal predictive accuracy. MSE(Best Model)=MSE from the top-ranking model; se=single-equation; me=multi-equation; *=lag length chosen by the BIC; HB=Hochberg-Bonferroni upper p-value bound for testing that all pairwise comparisons are zero; Maximum lag length=12 quarters.

Electronic copy available at: https://ssrn.com/abstract=3417527

Electronic copy available at: https://ssrn.com/abstract=3417527

Table A2: Remaining Out-of-Sample House Price Predictions - Part 2

	1 Quarter Ahead Growth Rate	1 Quarter Ahead Log Price	2 Quarters Ahead Growth Rate	2 Quarters Ahead Log Price	4 Quarters Ahead Growth Rate	4 Quarters Ahead Log Price	8 Quarters Ahead Growth Rate	8 Quarters Ahead Log Price
41	LSTM^{me} 1.023$^{[0.696]}$	LSTM^{me} 1.023$^{[0.696]}$	DecisionTree^{se} 1.108$^{[0.776]}$	RandomForest^{me} 1.061$^{[0.630]}$	RandomForest^{se} 1.259$^{[0.933]}$	RandomForest^{se} 1.229$^{[0.843]}$	RandomForest^{me} 1.200$^{[0.941]}$	RandomForest^{se} 1.554$^{[0.940]}$
42	DecisionTree^{se} 1.091$^{[0.742]}$	DecisionTree^{se} 1.091$^{[0.742]}$	RandomForest^{me} 1.184$^{[0.812]}$	DecisionTree^{me} 1.064$^{[0.836]}$	AR(0) 1.265$^{[0.866]}$	AR(0) 1.380$^{[0.865]}$	AR(0) 1.281$^{[0.876]}$	RandomForest^{me} 1.889$^{[0.947]}$
43	DecisionTree^{me} 1.091$^{[0.767]}$	DecisionTree^{me} 1.091$^{[0.767]}$	AR(0) 1.253$^{[0.856]}$	Ridge^{se} 1.165$^{[0.710]}$	RandomForest^{me} 1.434$^{[0.941]}$	RandomForest^{me} 1.382$^{[0.894]}$	SVR (Sig)^{me} 1.344$^{[0.930]}$	AR(0) 1.962$^{[0.936]}$
44	Ridge^{me} 1.158$^{[0.766]}$	Ridge^{me} 1.158$^{[0.766]}$	Ridge^{se} 1.363$^{[0.897]}$	AR(0) 1.294$^{[0.851]}$	Ridge^{me} 1.749$^{[0.995]}$	Ridge^{me} 1.648$^{[0.989]}$	Ridge^{me} 1.391$^{[0.946]}$	Ridge^{me} 2.171$^{[0.981]}$
45	AR(0) 1.228$^{[0.834]}$	AR(0) 1.228$^{[0.834]}$	Ridge^{me} 1.543$^{[0.990]}$	Ridge^{me} 1.347$^{[0.934]}$	Ridge^{se} 2.064$^{[0.993]}$	Ridge^{se} 1.878$^{[0.970]}$	Ridge^{se} 2.189$^{[0.992]}$	MLP^{me} 2.473$^{[0.966]}$
46	MLP^{me} 3.019$^{[1.000]}$	MLP^{me} 3.019$^{[1.000]}$	MLP^{me} 3.832$^{[1.000]}$	MLP^{me} 2.899$^{[1.000]}$	MLP^{me} 2.561$^{[1.000]}$	MLP^{me} 2.297$^{[0.996]}$	MLP^{me} 2.786$^{[0.998]}$	Ridge^{se} 4.587$^{[0.999]}$
47	VAR(12) 6.515$^{[1.000]}$	VAR(12) 6.515$^{[1.000]}$	VAR(12) 4.284$^{[1.000]}$	VAR(12) 4.347$^{[1.000]}$	VAR(12) 4.047$^{[1.000]}$	VAR(12) 3.235$^{[0.999]}$	VAR(12) 3.971$^{[0.998]}$	VAR(12) 5.297$^{[1.000]}$
HB (p-value)	[1.000]	[1.000]	[1.000]	[1.000]	[1.000]	[0.990]	[0.990]	[1.000]
MSE (Best Model)	2.426	2.426	2.613	8.307	3.056	29.479	3.124	62.611

Notes: Rankings according to MSEs relative to the benchmark random walk with drift model for log prices (historical average for growth rates). Square brackets contain p-values from DM/GW tests of equal predictive accuracy. MSE(Best Model)=MSE from the top-ranking model; se=single-equation; me=multi-equation; *=lag length chosen by the BIC; HB=Hochberg-Bonferroni upper p-value bound for testing that all pairwise comparisons are zero; Maximum lag length=12 quarters.